

TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ

KHOA CÔNG NGHỆ THÔNG TIN



THỰC TẬP ĐỒ ÁN CƠ SỞ NGÀNH

HỌC KỲ I, NĂM HỌC 2025-2026

Nhận dạng chữ số viết tay sử dụng thuật toán K - Nearest Neighbors (KNN)

Giảng viên hướng dẫn:

TS. Nguyễn Nhứt Lam

Sinh viên thực hiện:

Họ tên: Nguyễn Thành Trung

MSSV: 170122293

Vĩnh Long, tháng 5 năm 2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Vĩnh Long, ngày tháng năm

Giảng viên hướng dẫn

(Ký tên và ghi rõ họ tên)

NHẬN XÉT CỦA THÀNH VIÊN HỘI ĐỒNG

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Vĩnh Long, ngày tháng năm

Giảng viên hướng dẫn

(Ký tên và ghi rõ họ tên)

MỤC LỤC

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN	1
NHẬN XÉT CỦA THÀNH VIÊN HỘI ĐỒNG	2
MỤC LỤC	3
Danh mục từ viết tắt	5
Danh mục hình ảnh.....	6
Lời cảm ơn.....	7
TÓM TẮT ĐỒ ÁN	8
1. Vấn đề nghiên cứu.....	8
2. Hướng tiếp cận	8
3. Cách giải quyết vấn đề	8
4. Kết quả đạt được.....	9
MỞ ĐẦU	9
1. Lý do chọn đề tài	9
2. Mục đích nghiên cứu	10
3. Đối tượng nghiên cứu.....	10
3.1. Dữ liệu	10
3.2. Thuật toán.....	11
3.3. Kỹ thuật xử lý	11
3.4. Các chỉ số đánh giá mô hình.....	11
4. Phạm vi nghiên cứu	11
CHƯƠNG 1. TỔNG QUAN	12
1.1 GIỚI THIỆU TỔNG QUAN VỀ VẤN ĐỀ NGHIÊN CỨU	12
1.2 HƯỚNG NGHIÊN CỨU	12
1.2.1 Các bước thực hiện thuật toán KNN	13
1.2.1.1. Thu thập và Tiền xử lý dữ liệu (Data Loading & Preprocessing).....	13
1.2.1.2. Phân chia tập dữ liệu (Dataset Splitting).....	14
1.2.1.3. Thực nghiệm tìm siêu tham số K tối ưu (Hyperparameter Tuning).....	14
1.2.1.4. Huấn luyện và Thực thi thuật toán (Model Training)	14
1.2.1.5. Đánh giá kết quả chuyên sâu (Model Evaluation)	14
1.2.2 Các yếu tố ảnh hưởng đến hiệu quả của thuật toán KNN	15
1.2.2.1. Giá trị của tham số K (Số lượng láng giềng).....	15

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

1.2.2.2. Độ lớn và chất lượng của tập dữ liệu	16
1.2.2.3. Phương pháp đo khoảng cách (Distance Metric)	16
1.2.2.4. Kích thước không gian đặc trưng (Lời nguyên đa chiều)	16
1.2.2.5. Tiền xử lý và chuẩn hóa dữ liệu	16
CHƯƠNG 2. NGHIÊN CỨU LÝ THUYẾT	17
2.1 Cơ sở lý thuyết và lý luận.....	17
2.1.1 Tổng quan về học máy (Machine Learning)	17
2.1.2 Bài toán phân loại (Classification)	18
2.1.3 Thuật toán KNN và các phương pháp đo lường khoảng cách	18
2.1.4 Bộ dữ liệu MNIST	22
2.1.4.1 Tổng quan về Bộ dữ liệu MNIST.....	22
2.1.4.2 Triển khai thuật toán KNN cho nhận dạng chữ số	22
2.1.5 Các chỉ số đánh giá mô hình.....	23
2.2 Giả thiết khoa học.....	25
2.3 Công cụ, công nghệ và phần mềm sử dụng.....	27
2.3.1 Ngôn ngữ lập trình.....	27
2.3.2 Thư viện hỗ trợ và framework.....	27
2.3.3 Môi trường phát triển	30
2.3.3.1 Visual Studio Code (VS Code):	30
2.3.3.2 Venv (Python Virtual Environment)	31
2.3.4 Phần cứng	31
CHƯƠNG 3. HIỆN THỰC HÓA NGHIÊN CỨU	31
3.1. Quy trình nghiên cứu tổng thể.....	31
3.1.1 Thu thập và tìm hiểu dữ liệu.....	32
3.1.2 Tiền xử lý dữ liệu	32
3.1.3 Tối ưu hóa siêu tham số K.....	32
3.1.4 Huấn luyện và đánh giá mô hình.....	32
3.2 Thiết kế hệ thống	32
3.2.1 Sơ đồ tổng thể hệ thống.....	32
3.2.2 Thiết kế luồng xử lý (Flowchart).....	33
3.2.3 Lược đồ tiến trình thực hiện (Activity Diagram)	34
3.2.4 Mô hình dữ liệu	34
3.3. Cài đặt chương trình	35
3.3.1 Công nghệ sử dụng	35
3.3.2 Biên soạn và diễn giải cấu trúc code	38

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

3.3.2.1 Biên soạn code.....	38
3.3.2.2 Diễn giải cấu trúc code.....	38
3.3.2.3 Các bước thực hiện chạy code trên máy tính	45
3.4 Phân tích ý nghĩa, nhận xét và đánh giá hiệu quả.	48
3.4.1 Độ chính xác (Accuracy).....	49
3.4.2 Precision, Recall, F1-score	49
3.4.3 Ma trận nhầm lẫn.....	49
3.5. Kết luận phần hiện thực.....	52
CHƯƠNG 4. KẾT QUẢ NGHIÊN CỨU	52
4.1. Kết quả đạt được.....	52
4.2. Đánh giá hiệu năng mô hình.....	52
4.3. Phân tích ảnh hưởng của tham số K	53
4.4. Trực quan hóa kết quả	53
4.5. Trải nghiệm và tính ứng dụng	53
4.6. Nhận xét chung.....	54
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	54
5.1. Kết luận.....	54
5.2. Hướng phát triển.....	55
DANH MỤC TÀI LIỆU THAM KHẢO	55
1. Sách và Giáo trình	55
2. Bài báo Khoa học & Hội nghị	56
3. Tài liệu kỹ thuật & Website.....	57
PHỤ LỤC	57

Danh mục từ viết tắt

STT	Từ viết tắt	Ý nghĩa
1	AI	Artificial Intelligence
6	CMD	Command Prompt
7	CNN	Convolutional Neural Network

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

3	CPU	Central Processing Unit
4	KNN	K – nearest neighbor
2	ML	Machine learning
5	MNIST	Modified National Institute of Standards and Technology
8	NLP	Natural Language Processing

Danh mục hình ảnh

Hình 1.1 Thuật toán KNN	13
Hình 1.2 Bộ dữ liệu chữ số viết tay MNIST	13
Hình 1.3 Sơ đồ minh họa quy trình thực hiện	15
Hình 1.4 Hình minh họa ảnh hưởng của tham số K	17
Hình 2.1 Quá trình phân loại	18
Hình 2.2 Khoảng cách Euclid.....	19
Hình 2.3 Khoảng cách Manhattan	20
Hình 2.4 Khoảng cách Minkowski.....	21
Hình 2.5 Cách tính Precision và Recall.....	24
Hình 2.6 Mối liên hệ giữa F1-Score, Precision và Recall.....	25
Hình 2.7 Những số dễ thể nhầm lẫn	27
Hình 3.1 Sơ đồ tổng thể hệ thống.....	33
Hình 3.2 Thiết kế luồng xử lý (Flowchart).....	33
Hình 3.3 Lược đồ tiến trình thực hiện (Activity Diagram)	34
Hình 3.4 Mô hình dữ liệu	35
Hình 3.5 Python (3.14.3)	35
Hình 3.6 Visual Studio Code (1.117)	36
Hình 3.7 Cài venv	36
Hình 3.8 Thư mục venv	36
Hình 3.9 Các tệp trong venv (bạn không cần quan tâm đến những tệp này)	37
Hình 3.10 Kích hoạt venv thành công	37
Hình 3.11 Cài thư viện cần thiết.....	37
Hình 3.12 Kiểm tra lại các thư viện đã được cài	38
Hình 3.13 Kết quả sau khi chạy code	47
Hình 3.14 Độ chính xác với K tương ứng	50
Hình 3.15 Biểu đồ nhiệt Ma trận nhầm lẫn (heatmap)	50

Hướng dẫn thực thi trên máy tính 3.1	46
--	----

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

Hướng dẫn thực thi trên máy tính 3.2.....	46
Đoạn code 3.1 Import thư viện.....	39
Đoạn code 3.2 Load dữ liệu MNIST	39
Đoạn code 3.3 Chia tập train/test	39
Đoạn code 3.4 hiển thị ảnh ngẫu nhiên và kết quả.....	40
Đoạn code 3.5 kết quả kiểm tra phân bố nhãn	41
Đoạn code 3.6 kết quả huấn luyện K.....	42
Đoạn code 3.7 Dự đoán dữ liệu test	42
Đoạn code 3.8 kết quả tính toán các chỉ số	43
Đoạn code 3.9 Vẽ Confusion Matrix và Nâng cấp thành heatmap	44
Đoạn code 3.10 Biểu đồ ma trận nhầm lẫn (có heatmap)	44
Đoạn code 3.11 Vẽ biểu đồ Accuracy vs K	45
Bảng đánh giá kết quả ma trận nhầm lẫn 1	51

Lời cảm ơn

Trong quá trình thực hiện đồ án với đề tài “*Nhận dạng chữ số viết tay sử dụng thuật toán K-Nearest Neighbors (KNN)*”, em đã nhận được rất nhiều sự giúp đỡ, hỗ trợ và động viên quý báu từ thầy cô, gia đình và bạn bè.

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến **giảng viên hướng dẫn** – người đã tận tình chỉ bảo, định hướng và góp ý cho em trong suốt quá trình thực hiện đề tài. Những kiến thức chuyên môn và kinh nghiệm quý báu của thầy là nền tảng giúp em hoàn thành đồ án này.

Em cũng xin chân thành cảm ơn **quý thầy cô trong khoa Công nghệ Thông tin** đã truyền đạt những kiến thức nền tảng và chuyên sâu trong suốt thời gian học tập tại trường. Đây chính là hành trang quan trọng giúp em có thể tiếp cận và thực hiện đề tài một cách hiệu quả.

Bên cạnh đó, em xin gửi lời cảm ơn đến **gia đình và bạn bè** đã luôn bên cạnh, động viên và hỗ trợ em cả về tinh thần lẫn vật chất trong suốt quá trình học tập và thực hiện đồ án.

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

Mặc dù đã cố gắng hết sức để hoàn thiện đồ án, tuy nhiên do hạn chế về thời gian và kiến thức, đề tài khó tránh khỏi những thiếu sót. Em rất mong nhận được sự thông cảm và những ý kiến đóng góp quý báu từ quý thầy cô để đồ án được hoàn thiện hơn.

Em xin chân thành cảm ơn!

TP. HCM, ngày 10 tháng 05 năm 2026

Sinh viên thực hiện

Nguyễn Thành Trung

TÓM TẮT ĐỒ ÁN

1. Vấn đề nghiên cứu

Đồ án tập trung giải quyết bài toán **nhận dạng chữ số viết tay**, một trong những bài toán nền tảng của lĩnh vực thị giác máy tính và học máy [3], [10]. Cụ thể, đề tài hướng đến việc xây dựng một hệ thống có khả năng phân loại chính xác các mẫu ảnh chữ số từ 0 đến 9 dựa trên bộ dữ liệu chuẩn **MNIST** [11], [25]. Vấn đề thực nghiệm chính là làm sao để tối ưu hóa hiệu suất nhận diện thông qua việc điều chỉnh các tham số kỹ thuật của mô hình [4], [16].

2. Hướng tiếp cận

Đồ án sử dụng phương pháp **Học có giám sát (Supervised Learning)** với thuật toán **K-Nearest Neighbors (KNN)** [3], [9]. Đây là một thuật toán dựa trên thực thể (instance-based learning), hoạt động bằng cách so sánh khoảng cách giữa dữ liệu mới và các dữ liệu đã biết trong tập huấn luyện để đưa ra quyết định phân loại thông qua cơ chế bỏ phiếu đa số [8], [24].

3. Cách giải quyết vấn đề

Quy trình giải quyết vấn đề được triển khai thực nghiệm qua các bước chặt chẽ trên ngôn ngữ lập trình Python [11], [12]:

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

- ❖ **Xử lý dữ liệu:** Tải bộ dữ liệu MNIST gồm 70.000 mẫu, thực hiện chuyển đổi ảnh về dạng vector 784 chiều và chia dữ liệu theo tỷ lệ 80% huấn luyện - 20% kiểm tra [11], [25].
- ❖ **Tối ưu hóa tham số:** Thay vì chọn một giá trị K ngẫu nhiên, đề án tiến hành thực nghiệm (Hyperparameter Tuning) bằng cách chạy vòng lặp thử nghiệm các giá trị K từ 1 đến 10 [3], [8].
- ❖ **Tiêu chí lựa chọn:** Sử dụng biểu đồ tương quan giữa K và độ chính xác (Accuracy) để xác định giá trị K tối ưu nhất (Best K) cho mô hình cuối cùng.
- ❖ **Đánh giá đa chiều:** Sử dụng các công cụ đo lường chuyên sâu như Precision, Recall, F1-score và Ma trận nhầm lẫn (Confusion Matrix) để phân tích chi tiết khả năng nhận diện của mô hình trên từng chữ số.

4. Kết quả đạt được

- ❖ **Về kỹ thuật:** Xây dựng thành công mô hình KNN có khả năng nhận diện chữ số với độ chính xác cao. Mô hình đã xác định được giá trị K tối ưu giúp tối đa hóa hiệu suất phân loại [1].
- ❖ **Về phân tích:** Thông qua **Heatmap của Ma trận nhầm lẫn**, đề án đã chỉ ra được những cặp chữ số dễ gây nhầm lẫn nhất (ví dụ: các số có đặc điểm hình học tương đồng), từ đó cung cấp cái nhìn trực quan về điểm mạnh và hạn chế của thuật toán [2], [24].
- ❖ **Về ứng dụng:** Toàn bộ quy trình từ huấn luyện đến xuất báo cáo được tự động hóa, kết quả chi tiết được xuất ra file Excel giúp dễ dàng lưu trữ và thống kê [14], [20].

MỞ ĐẦU

1. Lý do chọn đề tài

Trong bối cảnh chuyển đổi số, việc tự động nhận dạng thông tin từ dữ liệu hình ảnh đóng vai trò quan trọng trong nhiều lĩnh vực như tài chính, giáo dục, hành chính và thương mại...

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

Nhận dạng chữ số viết tay là một bài toán cơ bản trong thị giác máy tính, nhưng lại có tính ứng dụng cực kỳ cao vào:

- ❖ Nhận dạng số trên hóa đơn, biểu mẫu
- ❖ Tự động chấm điểm bài thi trắc nghiệm
- ❖ Nhận dạng mã bưu chính, số tài khoản,...

Tuy nhiên, chữ viết tay có tính đa dạng cao (khác nhau về nét, độ nghiêng, độ đậm), gây khó khăn cho việc xử lý.

Thuật toán **KNN** tuy đơn giản nhưng có điểm mạnh là:

- 🚦 Không cần huấn luyện phức tạp.
- 🚦 Dễ triển khai.
- 🚦 Hiệu quả tốt với dữ liệu nhỏ và bài toán cơ bản.

2. Mục đích nghiên cứu

Đề tài hướng đến các mục tiêu:

- ❖ Xây dựng hệ thống nhận dạng chữ số viết tay dựa trên **KNN**.
- ❖ Phân tích ảnh hưởng của các yếu tố đến hiệu suất mô hình:
 - Giá trị K
 - Khoảng cách (Euclidean, Manhattan)
 - Chuẩn hóa dữ liệu
- ❖ Đánh giá và tối ưu độ chính xác của mô hình.
- ❖ Rút ra điểm mạnh và yếu của **KNN** trong bài toán nhận dạng ảnh.

3. Đối tượng nghiên cứu

3.1. Dữ liệu

- ❖ Bộ dữ liệu chữ số viết tay MNIST
- ❖ Ảnh grayscale kích thước 28x28 pixel

3.2. Thuật toán

- ❖ K-Nearest Neighbors (KNN)

3.3. Kỹ thuật xử lý

- ❖ Tiền xử lý ảnh:
 - Chuẩn hóa dữ liệu (0–1)
 - Làm phẳng ảnh (flatten vector)
- ❖ Tính khoảng cách:
 - Euclidean Distance
 - Manhattan Distance

3.4. Các chỉ số đánh giá mô hình

- ❖ Confusion Matrix: Ma trận nhầm lẫn.
- ❖ Accuracy: Độ chính xác tổng thể.
- ❖ Precision: Độ chính xác trên các dự đoán dương.
- ❖ Recall: Độ nhạy / Tỷ lệ gọi nhớ.
- ❖ F1-score: Chỉ số cân bằng.

4. Phạm vi nghiên cứu

- ❖ Đề tài tập trung nghiên cứu và triển khai phương pháp nhận dạng chữ số viết tay dựa trên thuật toán K - Nearest Neighbors (KNN) trong khuôn khổ dữ liệu và kỹ thuật cơ bản của học máy. Cụ thể:
 - Nghiên cứu được giới hạn trong bài toán **phân loại chữ số viết tay từ 0 đến 9** sử dụng bộ dữ liệu chuẩn **MNIST**, bao gồm các ảnh xám kích thước 28x28 pixel [11], [25].
 - Tập trung vào các bước chính gồm: tiền xử lý dữ liệu (chuẩn hóa, vector hóa), xây dựng mô hình KNN, lựa chọn tham số K và đánh giá kết quả.
 - Việc đánh giá mô hình được thực hiện thông qua các chỉ số cơ bản như **độ chính xác (Accuracy)**, **độ nhạy (Recall)**, **độ chính xác dương (Precision)** và ma trận nhầm lẫn (**Confusion Matrix**).

❖ Tìm hiểu cơ bản về học máy:

- Dữ liệu ảnh đơn giản.
- Tập trung vào KNN.
- Thực hiện trên Python; các thư viện cần thiết như: scikit-learn, numpy, matplotlib, pandas, openpyxl, seaborn. [4] , [12] , [23]

CHƯƠNG 1. TỔNG QUAN

1.1 GIỚI THIỆU TỔNG QUAN VỀ VẤN ĐỀ NGHIÊN CỨU

Trong bối cảnh phát triển cực kỳ mạnh và nhanh đến khó tin của trí tuệ nhân tạo và học máy, việc tự động hóa quá trình xử lý và nhận dạng dữ liệu từ hình ảnh đang trở thành một nhu cầu vô cùng thiết yếu [2], [9]. Một trong những bài toán cơ bản nhưng có ý nghĩa thực tiễn cao trong lĩnh vực này là nhận dạng chữ số viết tay (từ 0-9) [11].

Chữ số viết tay xuất hiện phổ biến trong rất nhiều hoạt động như xử lý biểu mẫu, hóa đơn, bài thi, hồ sơ hành chính và các hệ thống nhập liệu. Tuy nhiên, do sự khác biệt trong cách viết của từng người (về nét chữ, độ nghiêng, độ lệch, kích thước và độ đậm), việc xây dựng một hệ thống có khả năng nhận dạng chính xác các chữ số này là một thách thức lớn [7], [17].

Bài toán ở đây là làm thế nào để máy tính có thể “hiểu” và phân loại chính xác các chữ số viết tay từ dữ liệu ảnh. Đây chính là một dạng bài toán phân loại trong học máy, trong đó mỗi hình ảnh đầu vào cần được gán vào một lớp tương ứng từ 0 đến 9 [3], [10].

1.2 HƯỚNG NGHIÊN CỨU

Để giải quyết vấn đề này, sinh viên sẽ tập trung nghiên cứu và áp dụng thuật toán **KNN** - một phương pháp học máy có giám sát, hoạt động dựa trên nguyên lý so sánh khoảng cách giữa các điểm dữ liệu. Thuật toán này sẽ phân loại một đối tượng mới dựa trên các **đối tượng có khoảng cách gần nhất** (hay còn gọi là “các hàng xóm gần nhất”) của nó trong không gian dữ liệu.



Hình 1.1 Thuật toán KNN

1.2.1 Các bước thực hiện thuật toán KNN

Quy trình thực hiện được xây dựng bám sát cấu trúc của mã nguồn, chia làm các giai đoạn từ xử lý dữ liệu đến đánh giá mô hình chuyên sâu:

1.2.1.1. Thu thập và Tiền xử lý dữ liệu (Data Loading & Preprocessing)

- **Thu thập dữ liệu:** Sử dụng bộ dữ liệu MNIST với 70,000 mẫu ảnh chữ số viết tay.



Hình 1.2 Bộ dữ liệu chữ số viết tay MNIST

- **Chuyển đổi dữ liệu:** Trích xuất các đặc trưng hình ảnh dưới dạng vector 784 chiều và chuyển nhãn (target) sang kiểu số nguyên để tối ưu tính toán.
- **Thông kê và Trực quan hóa:** Kiểm tra sự phân bố dữ liệu bằng thư viện **Pandas** và hiển thị các mẫu ảnh ngẫu nhiên để đánh giá chất lượng tập dữ liệu đầu vào. [14]

1.2.1.2. Phân chia tập dữ liệu (Dataset Splitting)

- **Tỷ lệ phân chia:** Sử dụng phương pháp Hold-out để chia dữ liệu thành hai tập biệt lập: Tập huấn luyện (80%) và Tập kiểm tra (20%).
- **Tính khách quan:** Việc chia dữ liệu giúp đảm bảo mô hình được đánh giá trên những dữ liệu hoàn toàn mới, tránh hiện tượng quá khớp (Overfitting).

1.2.1.3. Thực nghiệm tìm siêu tham số K tối ưu (Hyperparameter Tuning)

- **Vòng lặp thực nghiệm:** Tiến hành thử nghiệm thuật toán với các giá trị láng giềng K chạy từ 1 đến 10.
- **Đánh giá hiệu suất theo K:** Tính toán độ chính xác (Accuracy) cho từng giá trị K và vẽ biểu đồ tương quan **Accuracy vs K**.
- **Lựa chọn Best K:** Xác định giá trị K mang lại độ chính xác cao nhất để làm tham số chính thức cho mô hình.

1.2.1.4. Huấn luyện và Thực thi thuật toán (Model Training)

- **Tính toán khoảng cách:** Tại mỗi điểm dữ liệu mới, mô hình tính toán khoảng cách (mặc định là Euclidean) tới các điểm trong tập huấn luyện.
- **Tìm láng giềng:** Sắp xếp và chọn ra K điểm dữ liệu gần nhất.
- **Biểu quyết (Voting):** Nhãn của dữ liệu mới được xác định dựa trên đa số phiếu bầu từ K láng giềng đã chọn.

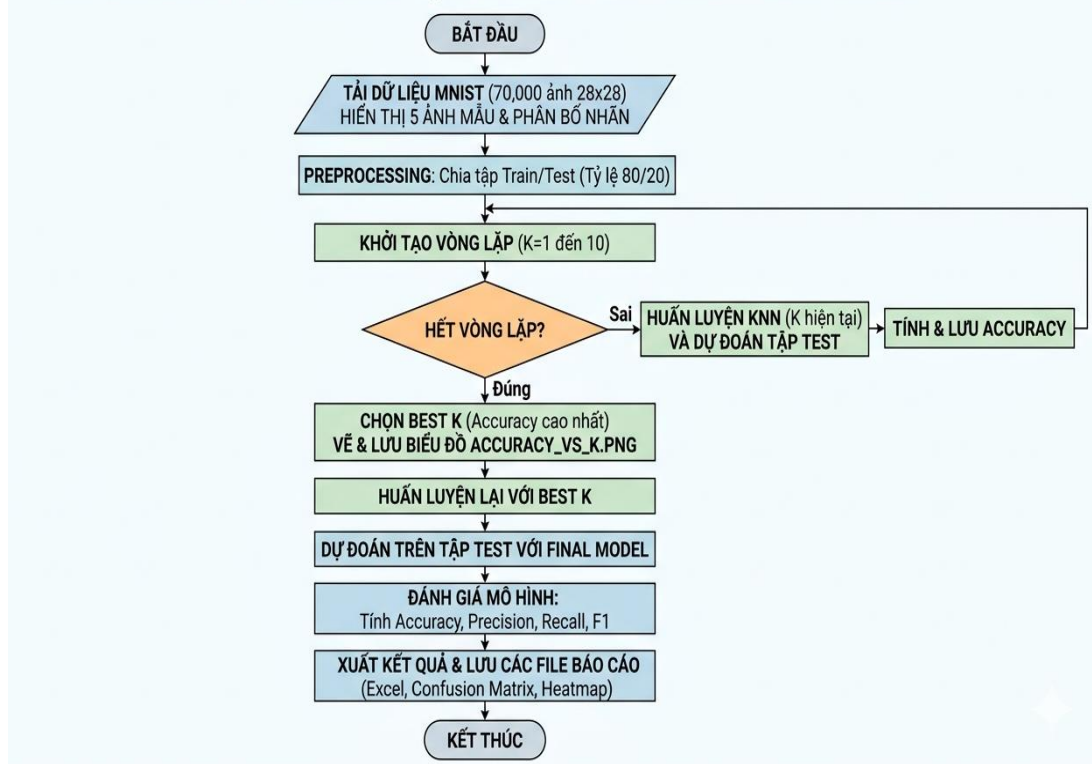
1.2.1.5. Đánh giá kết quả chuyên sâu (Model Evaluation)

- **Chỉ số định lượng:** Không chỉ dừng lại ở Accuracy, mô hình được đánh giá đa chiều qua các thông số: **Precision, Recall, và F1-score**.

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

- **Ma trận nhầm lẫn (Confusion Matrix):** Sử dụng Seaborn để vẽ Heatmap, giúp trực quan hóa các lỗi nhận diện giữa các cặp chữ số (ví dụ: số 4 và số 9).
- **Lưu trữ báo cáo:** Xuất toàn bộ báo cáo chi tiết (Classification Report) ra file Excel để phục vụ công tác thống kê và so sánh trong đồ án.

SƠ ĐỒ LƯỠNG: HUẤN LUYỆN VÀ ĐÁNH GIÁ KNN TRÊN MNIST



Hình 1.3 Sơ đồ minh họa quy trình thực hiện

1.2.2 Các yếu tố ảnh hưởng đến hiệu quả của thuật toán KNN

Hiệu suất của mô hình KNN không cố định mà phụ thuộc chặt chẽ vào cách thiết lập các thông số và đặc điểm của bộ dữ liệu. Các yếu tố chính bao gồm:

1.2.2.1. Giá trị của tham số K (Số lượng láng giềng)

Đây là yếu tố quan trọng nhất điều khiển sự cân bằng giữa nhiễu và độ chi tiết của mô hình:

- ❖ **K quá nhỏ (ví dụ $K = 1$):** Mô hình trở nên rất nhạy cảm với các điểm dữ liệu nhiễu (outliers). Điều này dễ dẫn đến hiện tượng **Overfitting** (quá khớp), nơi mô hình học thuộc lòng dữ liệu train thay vì tìm ra quy luật chung.

- ❖ **K quá lớn:** Mô hình sẽ lấy trung bình kết quả của quá nhiều điểm, làm mờ đi các ranh giới giữa các lớp chữ số. Điều này dẫn đến hiện tượng **Underfitting** (vị khớp), làm giảm độ chính xác trên các chi tiết đặc thù.

1.2.2.2. Độ lớn và chất lượng của tập dữ liệu

- ❖ **Số lượng mẫu:** KNN là thuật toán học dựa trên thực thể (instance-based). Càng có nhiều mẫu chữ số đa dạng trong tập huấn luyện, khả năng nhận diện các kiểu viết tay khác nhau càng cao.
- ❖ **Sự cân bằng giữa các lớp:** Nếu số lượng ảnh của số "1" quá nhiều so với số "9", mô hình sẽ có xu hướng thiên vị và dự đoán sai về phía lớp có đa số mẫu.

1.2.2.3. Phương pháp đo khoảng cách (Distance Metric)

Cách tính toán "độ gần" giữa hai vector ảnh 784 chiều ảnh hưởng trực tiếp đến kết quả tìm hàng xóm:

- ❖ **Euclidean Distance:** Phương pháp phổ biến nhất (mặc định trong Scikit-learn), tính khoảng cách theo đường thẳng [5], [6].
- ❖ **Manhattan Distance:** Tính khoảng cách theo các trục tọa độ, đôi khi hiệu quả hơn trong không gian nhiều chiều.
- ❖ **Minkowski Distance:** Dạng tổng quát của hai loại trên.

1.2.2.4. Kích thước không gian đặc trưng (Lời nguyên đa chiều)

Bộ dữ liệu MNIST có 784 đặc trưng (pixels). Trong không gian nhiều chiều như vậy, các điểm dữ liệu có xu hướng trở nên "xa" nhau hơn, khiến khái niệm "láng giềng gần nhất" bị giảm đi ý nghĩa.

- ❖ **Ảnh hưởng:** Làm tăng thời gian tính toán và có thể làm giảm độ chính xác nếu các pixels không chứa thông tin hữu ích (vùng trắng rìa ảnh).

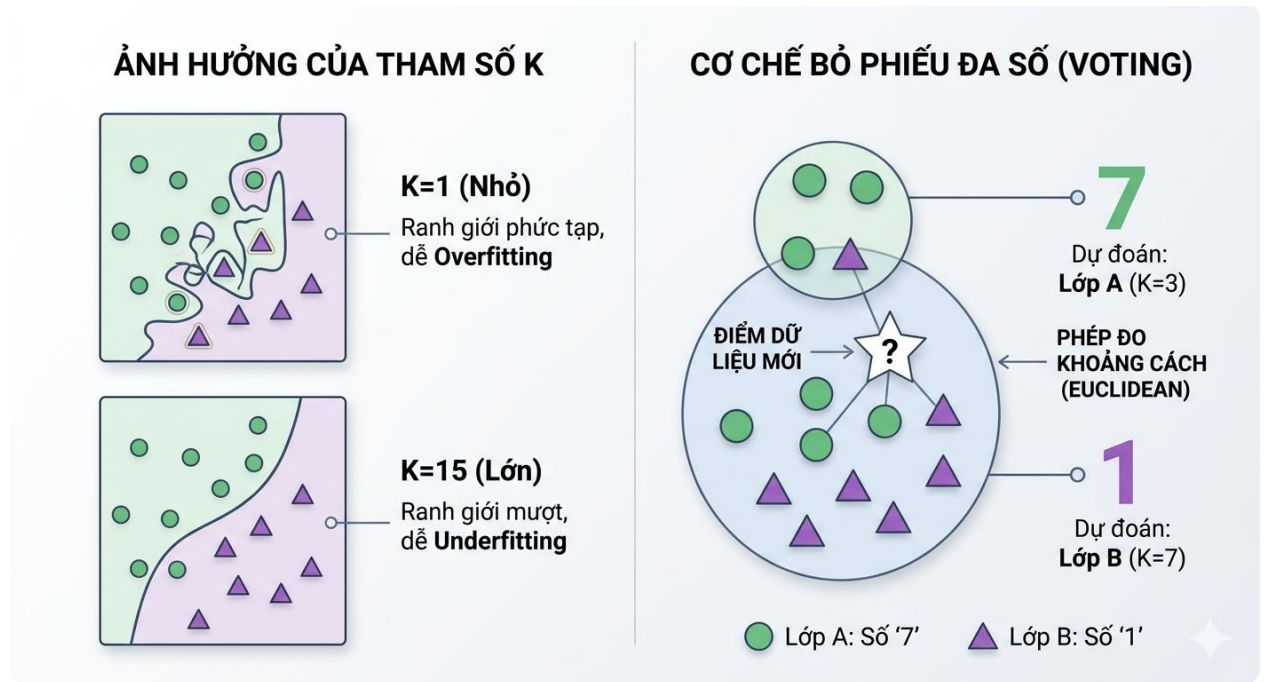
1.2.2.5. Tiền xử lý và chuẩn hóa dữ liệu

- ❖ Vì KNN dựa trên khoảng cách, nếu các đặc trưng có thang đo khác nhau (ví dụ: một cột từ 0-1, một cột từ 0-255), cột có giá trị lớn sẽ chiếm ưu thế hoàn toàn.

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

- ❖ **Chuẩn hóa (Normalization):** Đưa tất cả giá trị pixel về cùng một khoảng (thường là 0 đến 1) sẽ giúp mô hình hội tụ tốt hơn và công bằng hơn khi tính toán khoảng cách.

Trong phạm vi đề án này, yếu tố tham số K và chất lượng dữ liệu được tập trung tối ưu hóa thông qua các bước thực nghiệm tìm Best K và trực quan hóa mẫu ảnh đầu vào.



Hình 1.4 Hình minh họa ảnh hưởng của tham số K

CHƯƠNG 2. NGHIÊN CỨU LÝ THUYẾT

2.1 Cơ sở lý thuyết và lý luận

2.1.1 Tổng quan về học máy (Machine Learning)

Học máy là một nhánh của AI (Trí tuệ nhân tạo), cho phép máy tính học từ dữ liệu và cải thiện hiệu suất mà không cần lập trình tường minh. Trong đề tài này, bài toán được xem là **bài toán học có giám sát (Supervised Learning)** [10], trong đó mô hình được huấn luyện từ dữ liệu đã gán nhãn (các ảnh chữ số tương ứng với nhãn từ 0 đến 9).

Ngoài ra, học sâu (Deep Learning), một phân nhánh của học máy, là cấp độ phức tạp và tiên tiến hơn, sử dụng mạng neuron (nơ-ron) nhân tạo CNN (Convolutional Neural Network) nhiều lớp để xử lý dữ liệu lớn và giải quyết các bài toán học phức tạp. Học sâu đặc biệt hiệu quả trong các lĩnh vực nhận diện hình ảnh, giọng nói và Natural

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

Language Processing - Xử lý ngôn ngữ tự nhiên (NLP). Các công nghệ như xe tự lái, nhận diện khuôn mặt và dịch ngôn ngữ tự động đều dựa trên học sâu để hoạt động.

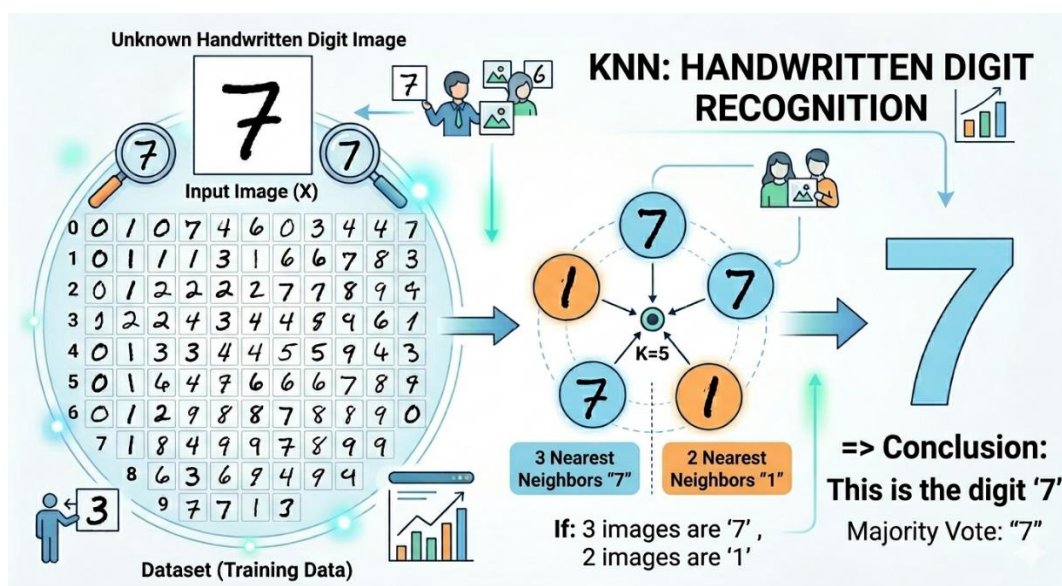
2.1.2 Bài toán phân loại (Classification)

Phân loại là một trong những bài toán cơ bản nhất của học máy, nhằm gán một đối tượng vào một trong các lớp đã xác định trước. Hay nói cách khác là gán một nhãn (label) cho dữ liệu mới dựa trên dữ liệu đã biết. Trong phạm vi đề tài này, bài toán phân loại là **xác định** chữ số (0–9) từ ảnh đầu vào.

Ví dụ dễ hiểu:

Bạn có một ảnh chữ số viết tay chưa biết là số gì. Áp dụng vào KNN sẽ làm như sau:

1. So sánh ảnh này với hàng nghìn ảnh trong dataset
2. Lấy ra K ảnh giống nhất
3. Nếu: 3 ảnh là số '7', 2 ảnh là số '1'
=> Kết luận: **đây là số '7'**



Hình 2.1 Quá trình phân loại

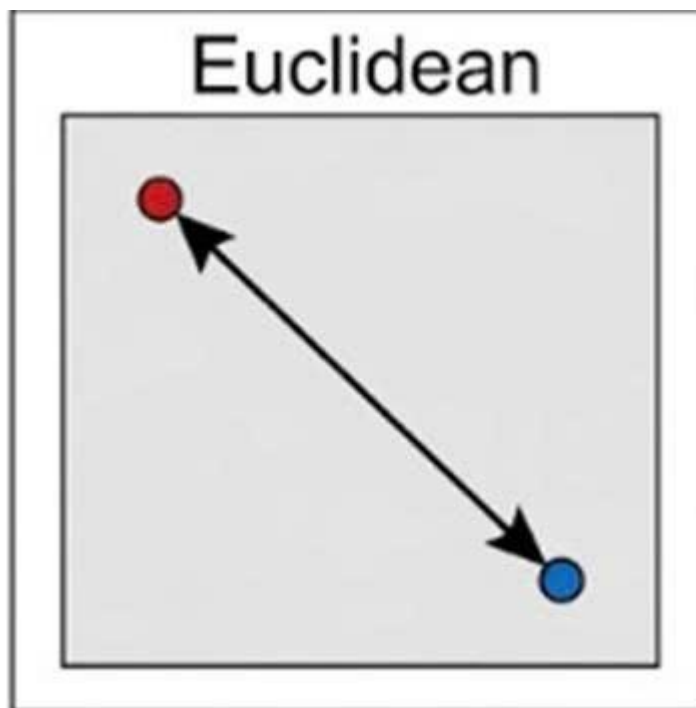
2.1.3 Thuật toán KNN và các phương pháp đo lường khoảng cách

KNN là một trong những thuật toán học máy có giám sát (supervised learning) đơn giản và phổ biến nhất. Nó dựa trên nguyên lý "**lòng tin theo số đông**" hoặc "**tính tương đồng về đặc điểm**". Trong KNN, một điểm dữ liệu mới sẽ được phân loại dựa trên khoảng cách của nó với các điểm dữ liệu đã biết trong tập huấn luyện.

❖ **Đặc điểm cốt lõi:**

- ✚ **Lazy Learning (Học lười):** Thuật toán không có giai đoạn huấn luyện tường minh để tạo ra một hàm mục tiêu. Thay vào đó, nó lưu trữ toàn bộ dữ liệu huấn luyện và chỉ thực hiện tính toán khi có dữ liệu mới cần dự báo [16], [24].
- ✚ **Non-parametric (Phi tham số):** KNN không đưa ra giả định nào về sự phân phối của dữ liệu gốc, giúp nó linh hoạt với các tập dữ liệu phức tạp trong thực tế.
- ✚ Một điểm dữ liệu mới sẽ được phân loại dựa trên **K điểm gần nhất** trong tập dữ liệu huấn luyện.
- ✚ Khoảng cách được sử dụng phổ biến nhất là **khoảng cách Euclid** (Euclidean distance).
 - Công thức tính **khoảng cách Euclid ($p = 2$):**

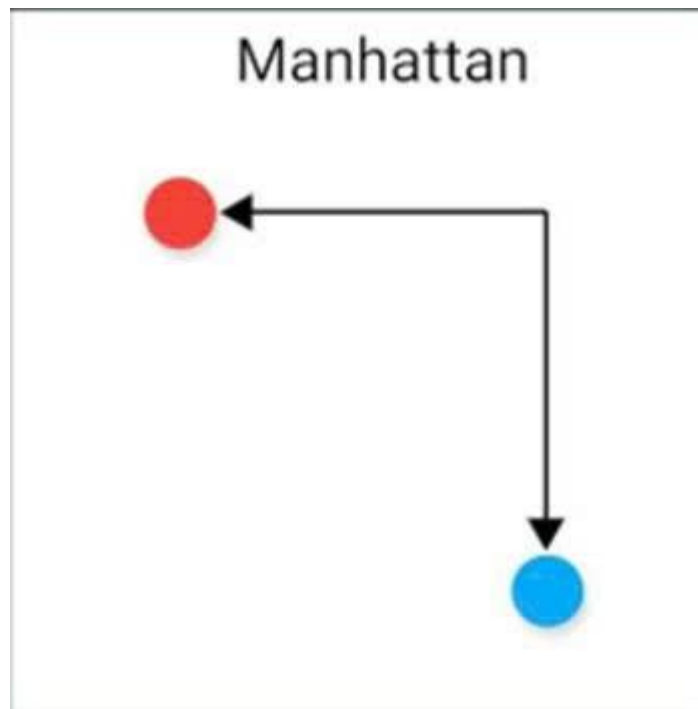
$$d(x, y) = \sqrt{\sum_{i=1}^n (y_i - x_i)^2}$$



Hình 2.2 Khoảng cách Euclid

- **Khoảng cách Manhattan ($p = 1$):** Đây cũng là một thước đo khoảng cách phổ biến, tính tổng giá trị tuyệt đối của sự chênh lệch giữa các tọa độ của hai điểm. Phương pháp này còn được gọi là khoảng cách taxi hoặc khoảng cách ô phố vì thường được hình dung trên một lưới đường phố, giống như cách chúng ta di chuyển từ địa chỉ này đến địa chỉ khác theo các con đường trong thành phố.

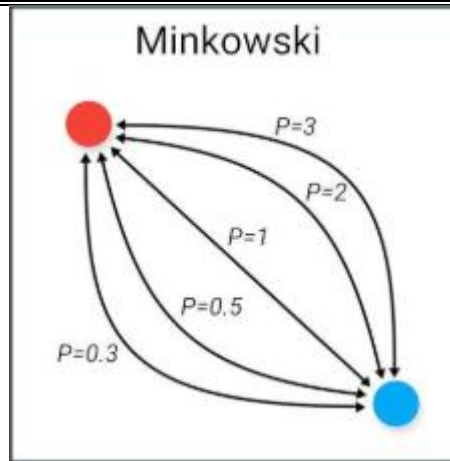
$$d(x, y) = \left(\sum_{i=1}^m |x_i - y_i| \right)$$



Hình 2.3 Khoảng cách Manhattan

- **Khoảng cách Minkowski:** Đây là một dạng tổng quát của cả khoảng cách Euclid và khoảng cách Manhattan. Tham số p trong công thức của Minkowski cho phép tạo ra nhiều thước đo khoảng cách khác nhau. Cụ thể, khi $p = 2$, công thức trở thành **khoảng cách Euclid**; còn khi $p = 1$, nó là **khoảng cách Manhattan**.

$$d(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}$$



Hình 2.4 Khoảng cách Minkowski

**Trong đồ án KNN này chúng ta dùng khoảng cách Euclid là mặc định của `KNeighborsClassifier`. Tức là nó tự tính khoảng cách giữa các vector ảnh (784 chiều của MNIST) để tìm K hàng xóm gần nhất.*

Tầm quan trọng của tham số K và Tiền xử lý dữ liệu:

❖ Lựa chọn giá trị K

- Nếu **K quá nhỏ** (ví dụ $K=1$): Mô hình dễ bị nhiễu (noise) tác động, dẫn đến hiện tượng **Overfitting** (quá khớp).
- Nếu **K quá lớn**: Ranh giới phân loại giữa các lớp bị mờ nhạt, dẫn đến **Underfitting** (chưa khớp).
- Thông thường, người ta chọn K là số lẻ để tránh tình trạng hòa nhau khi bình chọn.

Ưu điểm	Nhược điểm
➤ Dễ hiểu, dễ cài đặt, giải thích	➤ Tốn thời gian khi dự đoán với dữ liệu lớn
➤ Không cần quá trình huấn luyện phức tạp	➤ Nhạy cảm với nhiễu và dữ liệu không cân bằng
➤ Độ chính xác cao nếu tập dữ liệu đủ lớn và sạch.	➤ Đòi hỏi bộ nhớ lớn để lưu trữ toàn bộ tập huấn luyện.

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

➤ Hiệu quả với dữ liệu đa lớp.	➤ Gặp khó khăn với dữ liệu có số chiều lớn
--------------------------------	--

2.1.4 Bộ dữ liệu MNIST

2.1.4.1 Tổng quan về Bộ dữ liệu MNIST

Việc áp dụng KNN vào bộ dữ liệu MNIST là một bài toán kinh điển để minh chứng cho sức mạnh của việc phân loại dựa trên độ tương đồng hình học. MNIST là bộ dữ liệu chuẩn trong lĩnh vực nhận dạng ảnh, bao gồm:

- ❖ Bao gồm 70.000 mẫu chữ số viết tay từ 0 đến 9.
- ❖ Mỗi mẫu là một ảnh xám (grayscale) kích thước 28×28 pixel.
- ❖ Dữ liệu đầu vào: Mỗi pixel có giá trị từ 0 (màu đen/nền) đến 255 (màu trắng/nét chữ). Khi đưa vào KNN, ma trận 2D này thường được “trải phẳng” (flatten) thành một vector hàng có $28 \times 28 = 784$ chiều.

Dữ liệu đã được chuẩn hóa và gán nhãn sẵn, phù hợp cho các bài toán phân loại cơ bản.

2.1.4.2 Triển khai thuật toán KNN cho nhận dạng chữ số

Cơ chế so khớp đặc trưng.

Trong bài toán này, KNN hoạt động bằng cách so sánh vector 784 chiều của ảnh cần nhận dạng với tất cả các vector trong tập huấn luyện.

- **Không gian vector:** Mỗi ảnh là một điểm trong không gian 784 chiều.
- **Độ tương đồng:** Các ảnh có cách viết số giống nhau (ví dụ cùng là số "7") sẽ nằm gần nhau hơn trong không gian này so với các số khác (như số "8" hay "0").

Các bước triển khai cụ thể

- **Tiền xử lý (Preprocessing):**
 - **Rescaling:** Chuyển đổi giá trị pixel từ $[0, 255]$ về đoạn $[0, 1]$ để tăng tốc độ tính toán và đảm bảo tính ổn định.

- **Giảm chiều dữ liệu (Tùy chọn):** Có thể sử dụng PCA (Principal Component Analysis) để giảm từ 784 chiều xuống thấp hơn nhằm tối ưu thời gian chạy của KNN.
- **Lựa chọn tham số:**
 - Sử dụng **Khoảng cách Euclidean** là phổ biến nhất để đo độ lệch giữa các pixel tương ứng.
 - Chọn K (thường là K=3 hoặc K=5).
- **Phân loại:** Tìm K ảnh trong tập huấn luyện giống ảnh đầu vào nhất. Nhãn của ảnh đầu vào sẽ được quyết định bởi số đông (Majority Voting).

2.1.5 Các chỉ số đánh giá mô hình

Để đánh giá một cách khoa học trong luận án, ta cần sử dụng **Ma trận nhầm lẫn (Confusion Matrix)** và các chỉ số sau:

❖ **Confusion Matrix: Ma trận nhầm lẫn**

Trong bài toán phân loại, ma trận này thể hiện số lượng dự đoán đúng và sai của mô hình so với thực tế. Với bài toán nhị phân (ví dụ: số 7 hay không phải số 7), chúng ta có 4 giá trị:

- **TP (True Positive):** Dự đoán là 7, thực tế là 7 (Đúng).
- **TN (True Negative):** Dự đoán không phải 7, thực tế không phải 7 (Đúng).
- **FP (False Positive):** Dự đoán là 7, thực tế không phải 7 (Sai - Dương tính giả).
- **FN (False Negative):** Dự đoán không phải 7, thực tế là 7 (Sai - Âm tính giả).

❖ **Accuracy: Độ chính xác tổng thể.**

Tỉ lệ giữa số điểm dữ liệu được dự đoán đúng trên tổng số điểm dữ liệu.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- ➔ **Ý nghĩa:** Cho biết mô hình đoán đúng được bao nhiêu phần trăm trên toàn bộ tập dữ liệu.

❖ **Precision: Độ chính xác trên các dự đoán dương.**

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

Tỉ lệ số điểm thực sự là dương tính trong số những điểm được mô hình dự đoán là dương tính.

$$Precision = \frac{TP}{TP + FP}$$

➔ *Ý nghĩa:* Khi mô hình báo đây là "số 7", thì khả năng nó đúng là bao nhiêu? (Tránh việc dự đoán bừa bãi).

❖ **Recall: Độ nhạy / Tỉ lệ gọi nhớ.**

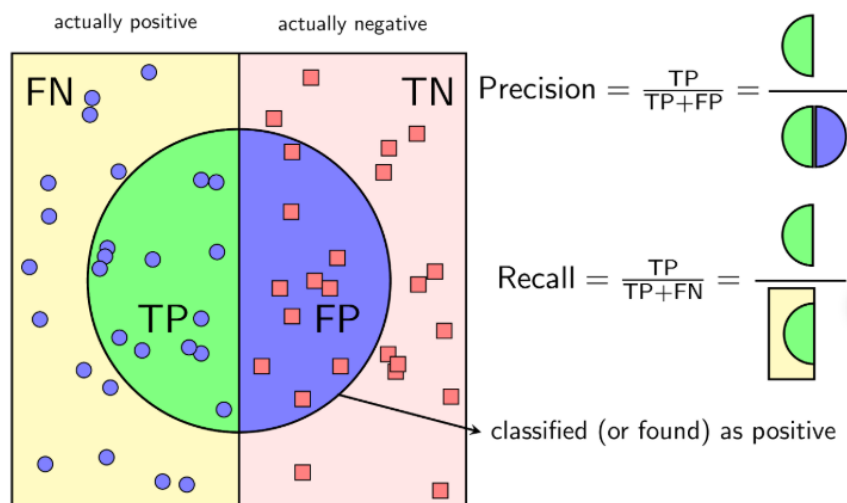
Tỉ lệ số điểm dương tính mà mô hình tìm thấy được trên tổng số điểm thực sự là dương tính.

$$Recall = \frac{TP}{TP + FN}$$

➔

Ý

nghĩa: Trong tất cả các "số 7" có trong dữ liệu, mô hình đã nhận diện được bao nhiêu số, tránh việc bỏ sót.



Hình 2.5 Cách tính Precision và Recall

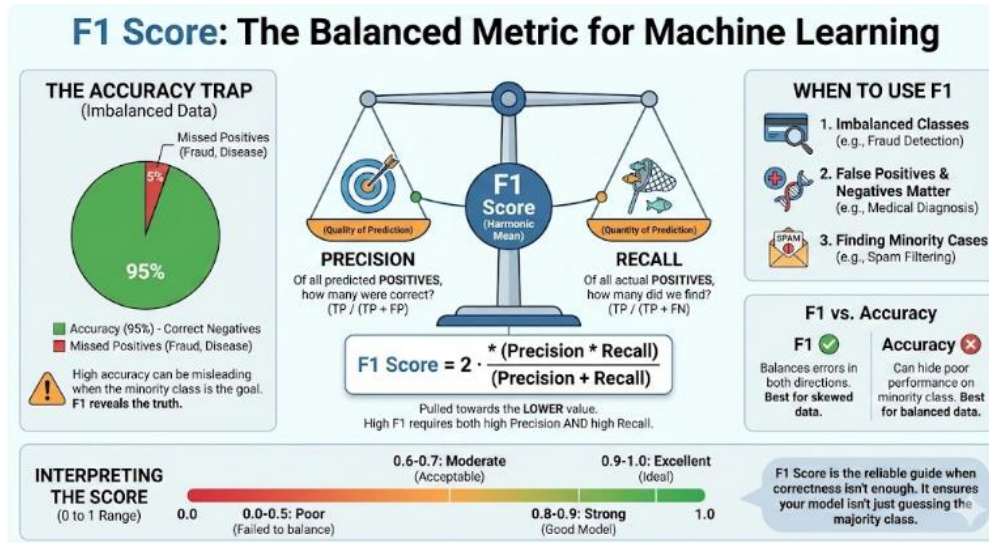
❖ **F1-score: Chỉ số cân bằng**

F1-score là trung bình điều hòa giữa Precision và Recall. Nó giúp ta có cái nhìn tổng quát khi Precision và Recall có sự chênh lệch lớn.

$$F1 - score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

→ *Ý nghĩa:* Một mô hình tốt cần có cả Precision và Recall cao, khi đó F1-score sẽ cao.



Hình 2.6 Mối liên hệ giữa F1-Score, Precision và Recall

2.2 Giả thiết khoa học

Đề tài được xây dựng dựa trên giả thiết: “Thuật toán K - Nearest Neighbors (KNN) có thể đạt được độ chính xác cao trong bài toán nhận dạng chữ số viết tay khi áp dụng trên bộ dữ liệu MNIST, nếu lựa chọn tham số K phù hợp và thực hiện tiền xử lý dữ liệu hợp lý.”

Nếu sử dụng thuật toán KNN với khoảng cách phù hợp (ví dụ: Euclidean distance) trên bộ dữ liệu MNIST đã được tiền xử lý (chuẩn hóa, giảm nhiễu), thì mô hình có thể phân loại chính xác các chữ số viết tay với độ chính xác cao ($\geq 90\%$).

Giả thiết này dựa trên cơ sở:

- ❖ MNIST là bộ dữ liệu **đã được chuẩn hóa tốt**, nghĩa là **các chữ số viết tay** trong bộ dữ liệu MNIST có **đặc trưng hình ảnh tương đối giống nhau trong cùng một lớp**.
- ❖ KNN hoạt động hiệu quả với dữ liệu có cấu trúc rõ ràng, Các điểm dữ liệu gần nhau trong không gian đặc trưng có xu hướng thuộc cùng một lớp.
- ❖ Khoảng cách giữa các điểm dữ liệu có thể phản ánh sự tương đồng giữa các chữ số. Cụ thể hơn chữ số ‘3’ sẽ gần với các chữ số ‘3’ khác hơn là ‘8’ hoặc ‘5’

Điều kiện để giả thiết đúng

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

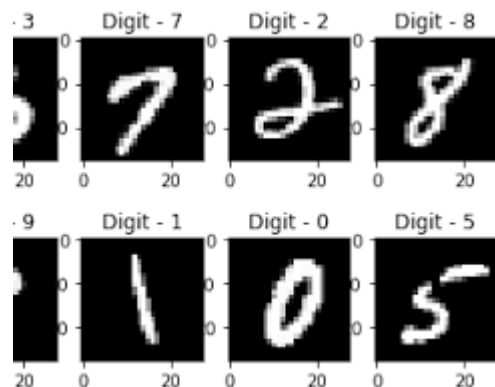
- ❖ Dữ liệu được:
 - Chuẩn hóa (normalize pixel)
 - Làm phẳng ảnh (flatten $28 \times 28 \rightarrow$ vector)
- ❖ Chọn tham số K phù hợp (thường $K = 3, 5, 7$)
- ❖ Sử dụng độ đo khoảng cách hợp lý (Euclidean hoặc Manhattan)

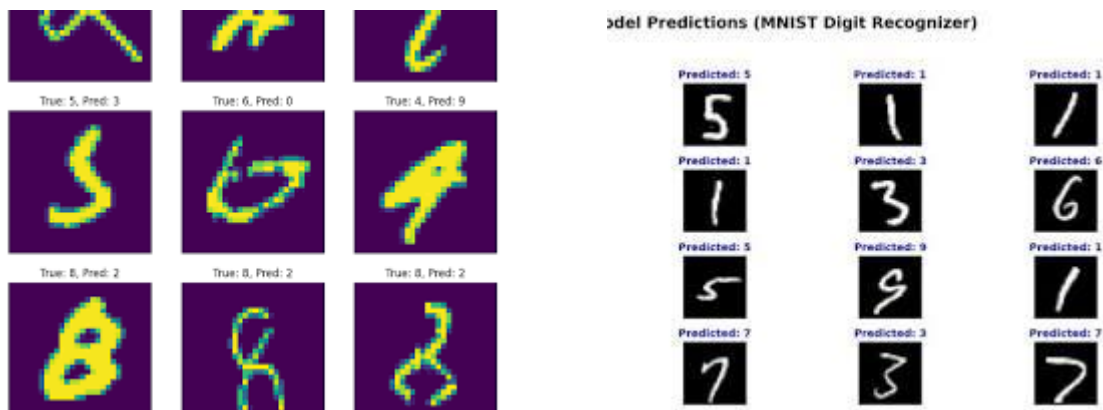
Biến độc lập và phụ thuộc

- ❖ Biến độc lập:
 - Giá trị K
 - Loại khoảng cách
 - Cách tiền xử lý dữ liệu
- ❖ Biến phụ thuộc:
 - Độ chính xác (Accuracy)
 - Độ nhạy (Recall)
 - Độ chính xác theo lớp (Precision)
 - F1-score

Kỳ vọng kết quả

- ❖ Mô hình KNN có thể đạt:
 - Accuracy $\approx 95\% - 97\%$ trên MNIST
- ❖ Tuy nhiên:
 - Có thể nhầm lẫn giữa các chữ số giống nhau như: 3-5, 4-9, 7-1





Hình 2.7 Những số dễ thể nhầm lẫn

Nhận xét: Trong các hình ảnh trên, ta có thể thấy

- **3 và 5:** Do đường cong ở thân dưới rất giống nhau, nếu nét gạch ngang phía trên của số 5 bị viết cong hoặc dính liền, KNN sẽ dễ nhầm nó thành số 3.
- **4 và 9:** Nếu phần đỉnh của số 4 được viết hơi chụm lại, nó trông cực kỳ giống một vòng tròn của số 9.
- **7 và 1:** Một số người viết số 7 không có gạch ngang ở giữa, khiến nó trông như một đường chéo tương tự số 1 viết nghiêng.

Vì KNN dựa trên khoảng cách (độ tương đồng về pixel), nên khi các chữ số này bị viết tay một cách cầu thả hoặc không rõ nét, các "láng giềng gần nhất" của chúng thường thuộc về nhóm chữ số có hình dáng tương đương, dẫn đến sai sót.

2.3 Công cụ, công nghệ và phần mềm sử dụng

Để thực hiện đề tài, ta sử dụng các công cụ, công nghệ và phần mềm sau:

2.3.1 Ngôn ngữ lập trình

❖ **Python:** Ngôn ngữ chính được sử dụng trong toàn bộ quá trình xây dựng mô hình, nhờ tính đơn giản, dễ học và có hệ sinh thái mạnh về học máy.

Lý do lựa chọn:

- Cú pháp đơn giản, dễ học và phù hợp cho xử lý dữ liệu.
- Có hệ sinh thái thư viện mạnh phục vụ học máy và xử lý ảnh.
- Được sử dụng phổ biến trong nghiên cứu và ứng dụng AI.

2.3.2 Thư viện hỗ trợ và framework

❖ **NumPy:** Xương sống của tính toán số học, là thư viện nền tảng cho tính toán khoa học trong Python. [18]

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

Trong Machine Learning, dữ liệu (như hình ảnh chữ số) không được hiểu là hình ảnh thông thường mà là các ma trận số.

- **Ý nghĩa:** Cung cấp các cấu trúc dữ liệu mảng đa chiều (n - dimensional array) mạnh mẽ.
- **Nhiệm vụ:** Thực hiện các phép toán đại số tuyến tính (tính khoảng cách giữa các điểm dữ liệu - yếu tố cốt lõi của KNN).
 - Thay đổi hình dạng dữ liệu (reshape), ví dụ: biến ảnh 28x28 thành một vector phẳng 784 chiều để đưa vào mô hình.
- ❖ **Scikit-learn:** Thư viện Machine Learning tiêu chuẩn

Đây là thư viện quan trọng nhất để triển khai các thuật toán học máy truyền thống.

- **Ý nghĩa:** Cung cấp các công cụ hiệu quả để khai phá và phân tích dữ liệu.
- **Nhiệm vụ:**
 - **Triển khai KNN:** Chứa lớp `KNeighborsClassifier` giúp bạn huấn luyện và dự đoán mà không cần viết lại thuật toán từ đầu.
 - **Xử lý dữ liệu:** Chia tập dữ liệu thành các phần Train/Test (`train_test_split`).

Đánh giá: Cung cấp các hàm tính toán Accuracy (độ chính xác), Sensitivity/Recall và Confusion Matrix để đo lường hiệu quả mô hình

- ❖ **Matplotlib:** Công cụ trực quan hóa dữ liệu

Dữ liệu số sẽ rất khó hiểu nếu không được hình ảnh hóa.

- **Ý nghĩa:** Thư viện vẽ đồ thị và hiển thị hình ảnh 2D chuyên nghiệp.
- **Nhiệm vụ:**
 - Hiển thị các mẫu ảnh từ bộ dữ liệu MNIST để kiểm tra xem chữ số đó trông như thế nào.
 - Vẽ biểu đồ biểu diễn độ chính xác của mô hình dựa trên các giá trị K khác nhau để tìm ra K tối ưu.

- ❖ **Pandas:** cho phép đưa dữ liệu vào cấu trúc DataFrame.

Trong bài toán nhận diện chữ số viết tay (MNIST) bằng KNN, **Pandas** không phải là thư viện bắt buộc như NumPy hay Scikit-learn, nhưng nó đóng vai trò là một "quản lý dữ liệu" cực kỳ chuyên nghiệp.

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

Quản lý dữ liệu dưới dạng bảng (DataFrame)

Thay vì làm việc với các mảng số thuần túy trong NumPy, Pandas cho phép bạn đưa dữ liệu vào cấu trúc **DataFrame** (giống như một bảng Excel).

- **Ý nghĩa:** Giúp bạn dễ dàng quan sát mối quan hệ giữa các đặc trưng (pixel) và nhãn (số 0-9).
- **Ứng dụng:** Bạn có thể đặt tên cột cho từng pixel (từ pixel_1 đến pixel_784) để dữ liệu trở nên tường minh hơn.

Kiểm tra và làm sạch dữ liệu (Data Cleaning)

Trước khi đưa vào mô hình KNN, dữ liệu thường cần được kiểm tra kỹ lưỡng.

- **Nhiệm vụ:**
 - **Kiểm tra giá trị thiếu (Null/NaN):** KNN cực kỳ nhạy cảm với dữ liệu thiếu. Pandas giúp bạn phát hiện nhanh bằng lệnh `.isnull().sum()`.
 - **Thống kê mô tả:** Dùng `.describe()` để xem giá trị pixel có nằm trong khoảng 0-255 hay không, hoặc dùng `.value_counts()` để kiểm tra xem số lượng mẫu của từng chữ số (0, 1, 2...) có bị lệch nhau quá nhiều không (mất cân bằng dữ liệu).

Tiền xử lý và Biến đổi

Mặc dù NumPy có thể tính toán nhanh, nhưng Pandas lại mạnh hơn ở việc thao tác logic trên tập dữ liệu lớn.

- **Lọc dữ liệu:** Nếu bạn chỉ muốn huấn luyện mô hình để phân biệt hai số "0" và "1", Pandas giúp bạn lọc lấy các hàng tương ứng chỉ trong một dòng mã.
- **Gộp dữ liệu:** Dễ dàng kết hợp tập đặc trưng (X) và tập nhãn (y) để thực hiện các thao tác xáo trộn (shuffling) dữ liệu trước khi chia tập Train/Test.

➤ Seaborn:

Trong bài toán nhận dạng chữ viết tay bằng thuật toán KNN, Seaborn đóng vai trò then chốt trong việc đánh giá và trình bày kết quả:

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

- ❖ Trực quan hóa Ma trận nhầm lẫn (Confusion Matrix): Sử dụng hàm `sns.heatmap()` để biểu diễn ma trận nhầm lẫn dưới dạng biểu đồ nhiệt. Điều này giúp dễ dàng nhận diện những cặp chữ số thường xuyên bị mô hình dự đoán sai (ví dụ: số 4 hay bị nhầm với số 9).
- ❖ Phân tích phân phối dữ liệu: Giúp kiểm tra xem số lượng mẫu của các chữ số từ 0 đến 9 trong tập dữ liệu MNIST có đồng đều hay không thông qua biểu đồ đếm (`sns.countplot`).
- ❖ Đánh giá sự tương quan: Mặc dù MNIST là dữ liệu pixel, nhưng trong các bài toán Machine Learning khác, Seaborn giúp tìm ra mối quan hệ giữa các đặc trưng thông qua biểu đồ cặp (`sns.pairplot`).

➤ Openpyxl:

Trong quy trình xây dựng và đánh giá mô hình KNN, Openpyxl đóng vai trò là "cầu nối" giữa mã nguồn và báo cáo kết quả thực nghiệm: [20]

- ❖ Lưu trữ kết quả dự đoán: Xuất kết quả phân loại từ mô hình KNN đối với tập dữ liệu kiểm thử (test set) ra tệp Excel để thuận tiện cho việc kiểm tra thủ công hoặc đối chiếu.
- ❖ Quản lý thông số thực nghiệm: Tự động ghi lại các chỉ số đánh giá như độ chính xác (Accuracy), sai số (Error Rate) tương ứng với các giá trị K khác nhau (với K là số láng giềng gần nhất). Điều này giúp dễ dàng chọn ra tham số K tối ưu thông qua bảng số liệu thống kê.
- ❖ Tự động hóa báo cáo: Thay vì phải sao chép kết quả từ màn hình console, Openpyxl giúp tự động điền các báo cáo phân loại (Classification Report) vào các biểu mẫu báo cáo có sẵn, giảm thiểu sai sót con người.

2.3.3 Môi trường phát triển

2.3.3.1 Visual Studio Code (VS Code):

Là trình soạn thảo mã nguồn (code editor) miễn phí, mã nguồn mở, nhẹ và mạnh mẽ do Microsoft phát triển. Phần mềm hỗ trợ đa nền tảng (Windows, macOS, Linux) và phổ biến nhất hiện nay cho các lập trình viên để viết, chỉnh sửa và gỡ lỗi code (debug) trên nhiều ngôn ngữ lập trình khác nhau. [21] ,[22]

- ❖ Dùng để lập trình và quản lý mã nguồn.

- ❖ Hỗ trợ nhiều hướng mở rộng (extension) cho Python và Machine Learning.

2.2.3.2 Venv (Python Virtual Environment)

Là môi trường Python ảo, cho phép mỗi dự án sử dụng một hệ thống thư viện riêng biệt, độc lập với môi trường Python toàn cục của máy tính. Việc sử dụng venv giúp đảm bảo tính độc lập, ổn định và khả năng tái sử dụng của môi trường phát triển, từ đó nâng cao hiệu quả xây dựng và triển khai các dự án Python.

2.3.4 Phần cứng

❖ Yêu cầu:

- CPU: Intel Core i5 hoặc tương đương.
- RAM: tối thiểu 8GB.

❖ Do KNN không yêu cầu GPU, nên có thể chạy tốt trên máy tính thông thường.

Một số lý do khiến KNN phù hợp với máy tính thông thường (CPU-based):

- **Không cần huấn luyện phức tạp:** Khác với Deep Learning (mạng nơ-ron) cần GPU để tính toán hàng triệu tham số trong quá trình huấn luyện, KNN là một thuật toán "lười" (Lazy Learning). Nó không có giai đoạn huấn luyện thực sự mà chỉ lưu trữ dữ liệu.
- **Tính toán đơn giản:** Công việc chính của KNN là tính khoảng cách (như khoảng cách Euclidean). Các bộ vi xử lý (CPU) hiện đại hoàn toàn có thể xử lý tốt các phép toán này với các tập dữ liệu vừa và nhỏ.
- **Tính ứng dụng cao:** Đối với các bài toán có lượng dữ liệu không quá khổng lồ, việc triển khai KNN trên CPU giúp tiết kiệm chi phí phần cứng và dễ dàng cài đặt trên hầu hết các dòng laptop phổ thông.

CHƯƠNG 3. HIỆN THỰC HÓA NGHIÊN CỨU

3.1. Quy trình nghiên cứu tổng thể

Quá trình thực hiện đề tài được triển khai theo các bước sau:

3.1.1 Thu thập và tìm hiểu dữ liệu

- ❖ Sử dụng bộ dữ liệu **MNIST** gồm 70.000 ảnh chữ số viết tay (0–9) được tải qua thư viện **sklearn**.
- ❖ Phân tích đặc điểm dữ liệu đầu vào và hiển thị các mẫu ảnh ngẫu nhiên để kiểm tra tính toàn vẹn.

3.1.2 Tiền xử lý dữ liệu

- ❖ Chuyển đổi dữ liệu nhãn sang kiểu số nguyên và quản lý dữ liệu bằng cấu trúc **DataFrame**.
- ❖ Chia tập train/test theo tỷ lệ 80/20. Nghĩa là từ bộ MNIST chia 70.000 ảnh thành 56.000 ảnh dùng để huấn luyện, 14.000 ảnh dùng để kiểm tra.
- ❖ Sử dụng thư viện **Pandas** để quản lý và kiểm tra phân bố nhãn dữ liệu.
- ❖ Chuyển ảnh 28x28 thành vector 784 chiều.
- ❖ Gán nhãn tương ứng cho từng ảnh.

3.1.3 Tối ưu hóa siêu tham số K

- ❖ Thực hiện vòng lặp thử nghiệm tham số K từ 1 đến 10 để tìm giá trị tối ưu (Best K).
- ❖ Sử dụng thư viện **Scikit-learn** để triển khai thuật toán **KneighborsClassifier**

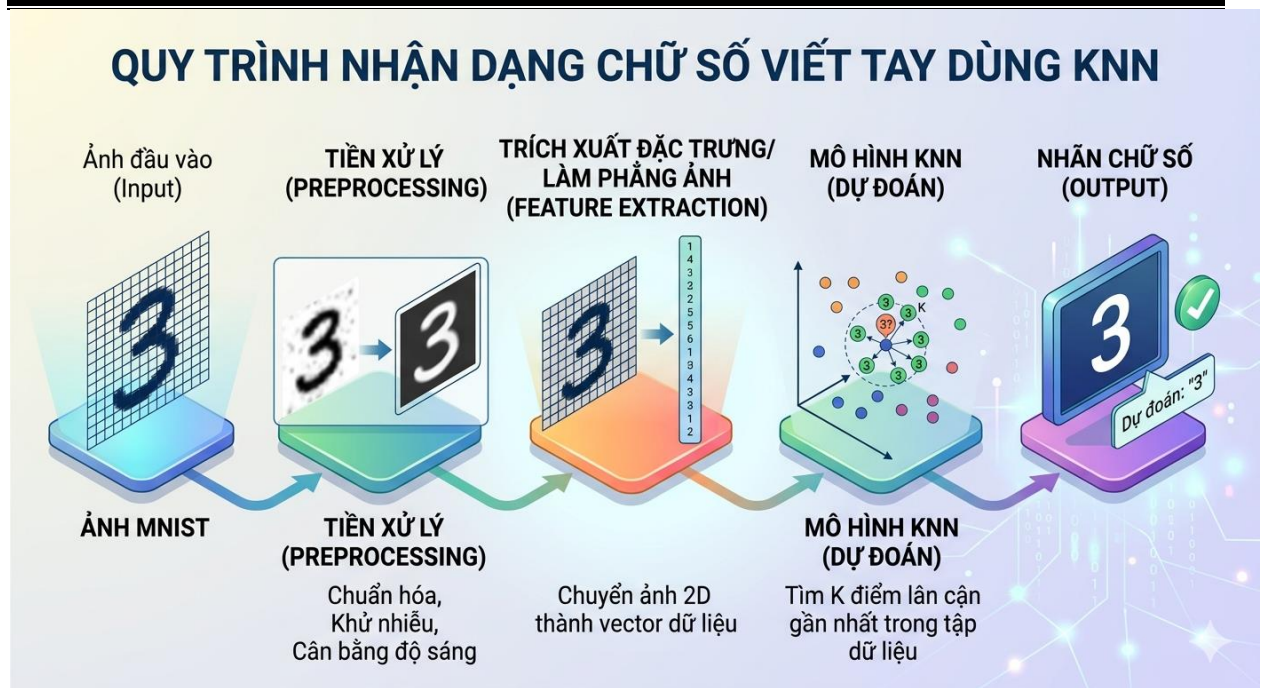
3.1.4 Huấn luyện và đánh giá mô hình

- ❖ Huấn luyện mô hình KNN với giá trị K tốt nhất đã tìm được.
- ❖ Đánh giá hiệu suất thông qua hệ chỉ số: Accuracy, Precision, Recall, F1-score.
- ❖ Trực quan hóa kết quả bằng Ma trận nhầm lẫn và biểu đồ Heatmap.
- ❖ Xuất báo cáo kết quả chi tiết ra file Excel phục vụ công tác thống kê.

3.2 Thiết kế hệ thống

3.2.1 Sơ đồ tổng thể hệ thống

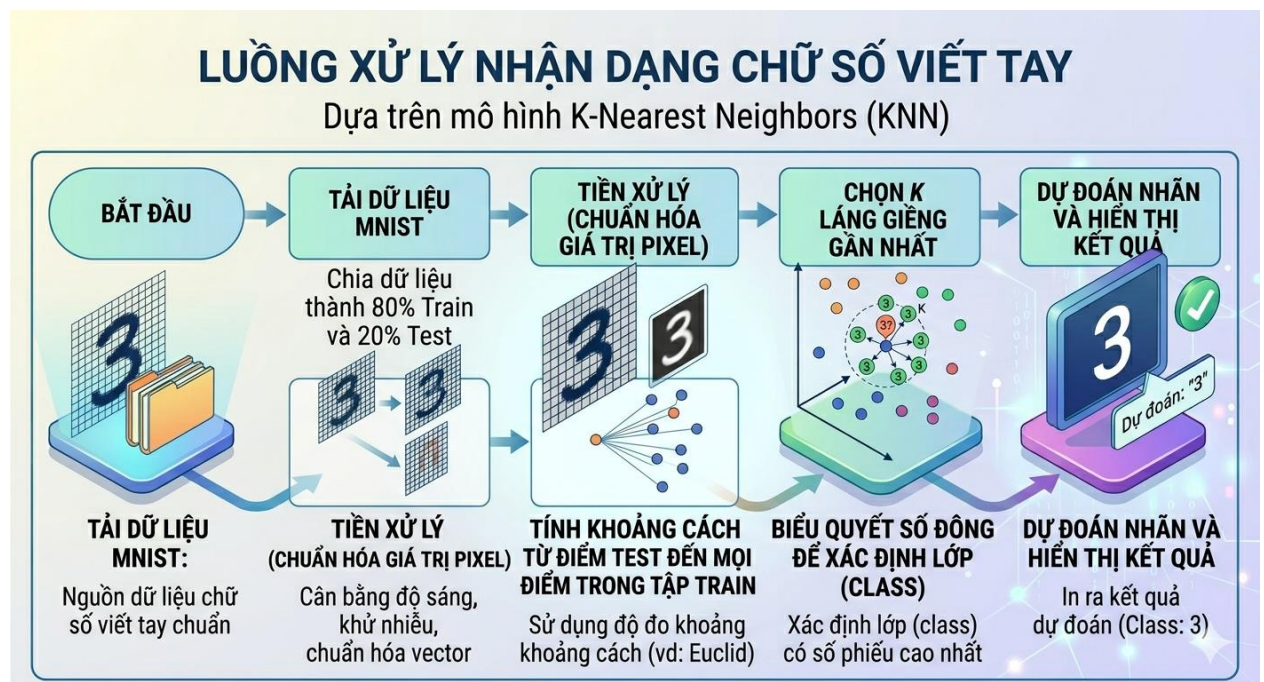
Input (Ảnh MNIST) → Preprocessing (Tiền xử lý) → Feature Extraction (Trích xuất đặc trưng/Làm phẳng ảnh) → KNN Model (Dự đoán) → Output (Nhãn chữ số).



Hình 3.1 Sơ đồ tổng thể hệ thống

3.2.2 Thiết kế luồng xử lý (Flowchart)

Bắt đầu → Tải dữ liệu MNIST → Tiền xử lý dữ liệu → Chia tập train/test → Tiền xử lý (Chuẩn hóa giá trị pixel) → Tính khoảng cách từ điểm dữ liệu Test đến mọi điểm trong tập Train → Chọn K láng giềng gần nhất → Biểu quyết số đông để xác định lớp (Class) → Kết thúc

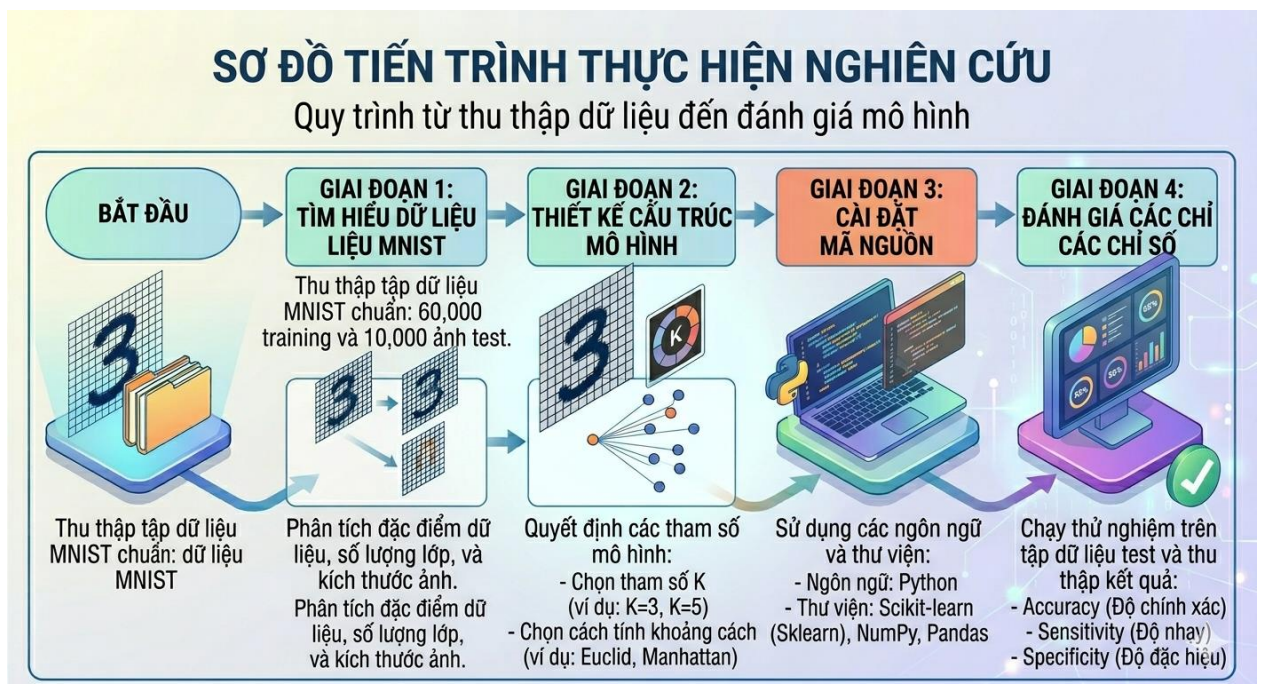


Hình 3.2 Thiết kế luồng xử lý (Flowchart)

3.2.3 Lược đồ tiến trình thực hiện (Activity Diagram)

❖ Sơ đồ này mô tả các giai đoạn bạn thực hiện nghiên cứu, từ lúc thu thập dữ liệu đến lúc đánh giá:

- Giai đoạn 1: Thu thập và tìm hiểu dữ liệu MNIST.
- Giai đoạn 2: Thiết kế cấu trúc mô hình (chọn tham số K, chọn cách tính khoảng cách).
- Giai đoạn 3: Cài đặt mã nguồn (sử dụng Python, Sklearn...).
- Giai đoạn 4: Chạy thử nghiệm và thu thập các chỉ số (Accuracy, Sensitivity...).



Hình 3.3 Lược đồ tiến trình thực hiện (Activity Diagram)

3.2.4 Mô hình dữ liệu

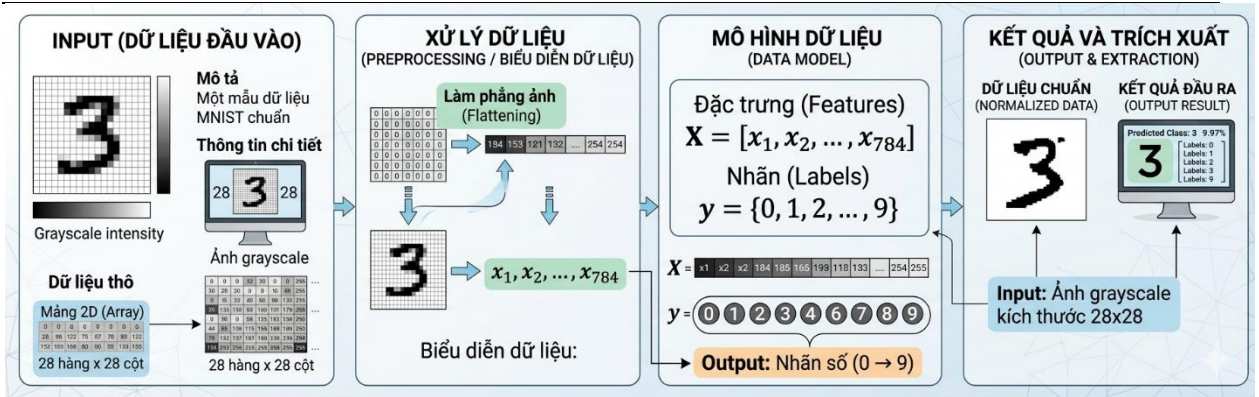
- Input: Ảnh grayscale kích thước 28x28
- Output: Nhãn số (0 → 9)

Biểu diễn dữ liệu:

$$X = [x_1, x_2, \dots, x_{784}]$$

$$y = \{0, 1, 2, \dots, 9\}$$

Nhận dạng chữ số viết tay sử dụng thuật toán KNN



Hình 3.4 Mô hình dữ liệu

3.3. Cài đặt chương trình

3.3.1 Công nghệ sử dụng

❖ Ngôn ngữ lập trình: Python

Cách cài đặt: Python (3.14.3)

link <https://www.python.org/downloads/release/python-3150a7/>

```
C:\Users\HP>py --version
Python 3.14.3

C:\Users\HP>
```

Hình 3.5 Python (3.14.3)

❖ Môi trường lập trình: VS Code

➤ VS Code

Cách cài đặt: Visual Studio Code (1.117)

link <https://code.visualstudio.com/download>

```
C:\Users\HP>code --version
1.117.0
10c8e557c8b9f9ed0a87f61f1c9a44bde731c409
x64

C:\Users\HP>|
```

Hình 3.6 Visual Studio Code (1.117)

➤ Venv (Python Virtual Environment)

Cách cài đặt: Venv (Python Virtual Environment).

Bước 1: Tại Desktop, tạo file chứa project.

Bước 2: Mở terminal trong VS Code: **Ctrl + `**

Bước 3: Di chuyển tới thư mục project.

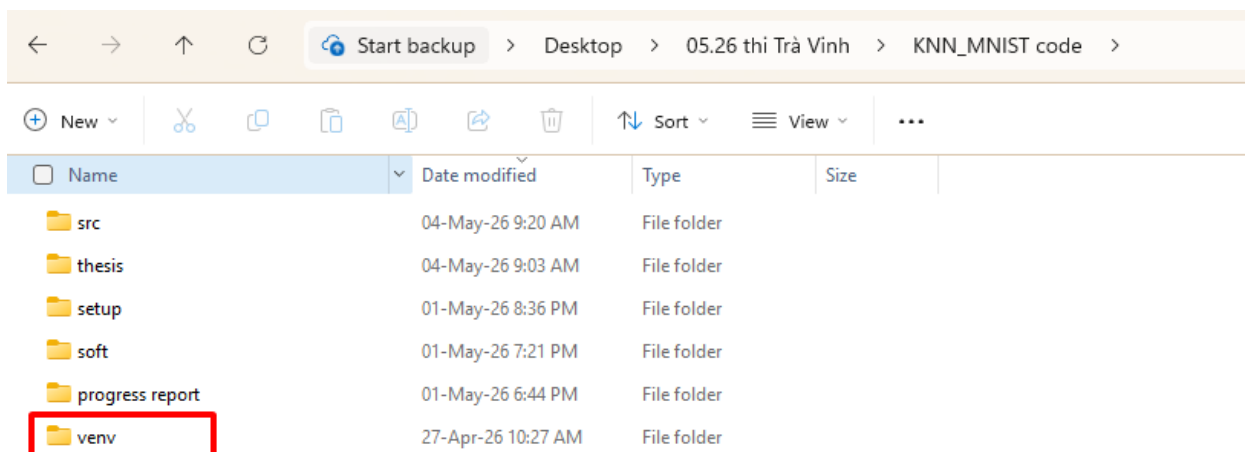
Bước 4: Tạo venv, nhập lệnh **py -m venv venv**

➔ Sau lệnh này sẽ có folder **venv**

Bước 5: Kích hoạt **venv** Nhập lệnh **venv\Scripts\Activate**

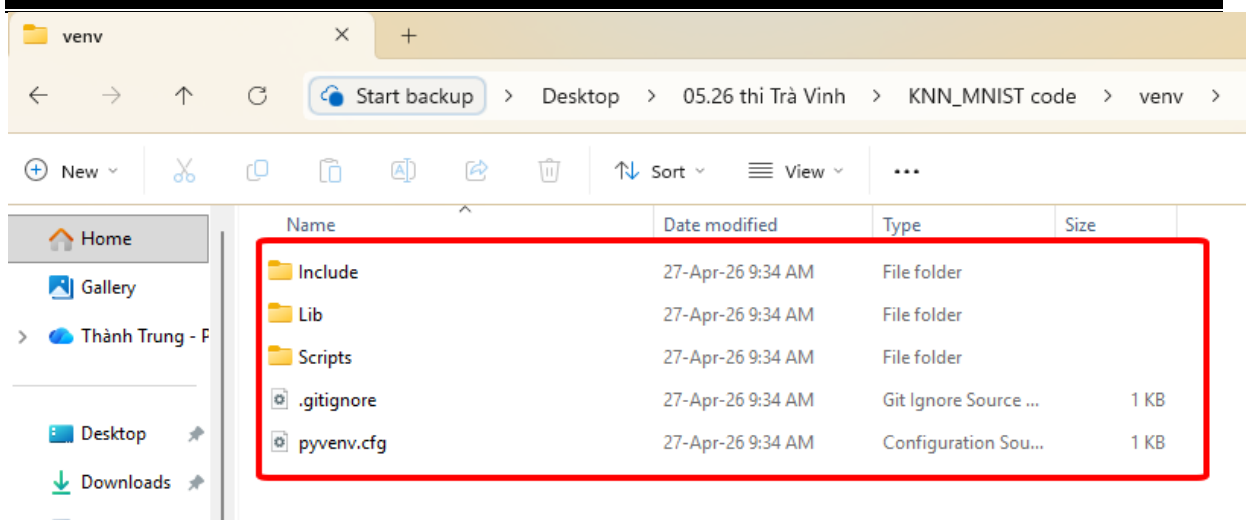
```
PS C:\Users\HP\Desktop\05.26 thi Trà Vinh\KNN_MNIST code> py -m venv venv
PS C:\Users\HP\Desktop\05.26 thi Trà Vinh\KNN_MNIST code> |
```

Hình 3.7 Cài venv



Hình 3.8 Thư mục venv

Nhận dạng chữ số viết tay sử dụng thuật toán KNN



Hình 3.9 Các tệp trong venv (bạn không cần quan tâm đến những tệp này)

```
PS C:\Users\HP\Desktop\05.26 thi Trà Vinh\KNN_MNIST code> venv\Scripts\Activate
(venv) PS C:\Users\HP\Desktop\05.26 thi Trà Vinh\KNN_MNIST code>
```

Hình 3.10 Kích hoạt venv thành công

❖ Các thư viện cần thiết

Bước 1: Tạo file **set up** → tạo một tệp **requirements.txt** để chứa các thư viện sau:

- NumPy
- Scikit-learn
- Matplotlib
- Pandas
- Seaborn
- Openpyxl

Bước 2: Sau khi Kích hoạt **venv** : **venv\Scripts\Activate**

Nhập lệnh cài đặt thư viện như sau: **pip install -r requirements.txt**

Bước 3: Kiểm tra lại các thư viện đã cài bằng lệnh: **pip list**

```
PS C:\Users\HP\Desktop\05.26 thi Trà Vinh\KNN_MNIST code> venv\Scripts\Activate
(venv) PS C:\Users\HP\Desktop\05.26 thi Trà Vinh\KNN_MNIST code> pip install -r requirements.txt
```

Hình 3.11 Cài thư viện cần thiết

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

```
(venv) PS C:\Users\HP\Desktop\05.26 thi Trà Vinh\KNN_MNIST code> pip list
Package            Version
-----
contourpy          1.3.3
cyclor              0.12.1
et_xmlfile         2.0.0
fonttools          4.62.1
joblib              1.5.3
kiwisolver          1.5.0
matplotlib          3.10.9
numpy               2.4.4
openpyxl            3.1.5
packaging           26.2
pandas              3.0.2
pillow              12.2.0
pip                 25.3
pyparsing           3.3.2
python-dateutil     2.9.0.post0
scikit-learn        1.8.0
scipy               1.17.1
seaborn             0.13.2
six                 1.17.0
threadpoolctl       3.6.0
tzdata              2026.2
```

Hình 3.12 Kiểm tra lại các thư viện đã được cài

3.3.2 Biên soạn và diễn giải cấu trúc code

3.3.2.1 Biên soạn code

Tại tệp `src` trong project → tạo `train.txt` và dán toàn bộ đoạn code từ link <https://bom.so/Ruz5wb> vào và save lại. Sau đó click chuột phải vào file, rename và đổi đuôi `.txt` thành đuôi `.py` → `train.py`

→ Như vậy là ta đã có một file code hoàn chỉnh để tiến hành thực hiện yêu cầu của đề án.

3.3.2.2 Diễn giải cấu trúc code

Cấu trúc các đoạn code

Bước 1: **Import thư viện:** Phần này bao gồm các thư viện để xử lý dữ liệu, mô hình KNN và vẽ biểu đồ, tự động tạo thư mục lưu trữ hình ảnh **thesis/abs** (nếu người thực hiện quên chưa tạo, máy sẽ khởi tạo giúp. Nếu có rồi, máy sẽ bỏ qua bước này).

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

```
import os
# Khai báo biến đường dẫn chung để dễ quản lý và thay đổi
output_path = "thesis/abs"
# máy sẽ tạo thư mục đầu ra nếu chưa có
if not os.path.exists(output_path):
    os.makedirs(output_path)
# import các thư viện cần thiết
from sklearn.datasets import fetch_openml          # Load dataset MNIST từ OpenML
from sklearn.model_selection import train_test_split # Chia dữ liệu train/test
from sklearn.neighbors import KNeighborsClassifier  # Thuật toán KNN
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay # Đán
from sklearn.metrics import classification_report, precision_score, recall_score, f1_score
import seaborn as sns                               # Vẽ biểu đồ nhiệt
import matplotlib.pyplot as plt                     # Vẽ biểu đồ
import numpy as np                                  # Xử lý mảng số
from collections import Counter                     # Đếm số lượng phần tử
import pandas as pd                                 # Xử lý dữ liệu dạng bảng
```

Đoạn code 3.1 Import thư viện

Bước 2: Load dữ liệu MNIST: Sử dụng `fetch_openml` để tải bộ dữ liệu MNIST nổi tiếng về máy.

```
# ===== LOAD DATA =====
print("--- Loading MNIST... ---")
mnist = fetch_openml('mnist_784', version=1) # Tải dataset MNIST (70,000 ảnh 28x28)

X = mnist.data                                # Dữ liệu ảnh (dạng vector 784 chiều)
y = mnist.target.astype(int)                  # Nhãn (0-9), chuyển về số nguyên

X = X / 255.0 # dòng này để chuẩn hóa dữ liệu về khoảng [0-1]
```

Đoạn code 3.2 Load dữ liệu MNIST

Bước 3: Chia tập train/test: Chia bộ dữ liệu theo tỷ lệ 80% để huấn luyện và 20% để kiểm tra.

```
# ===== CHIA DATA =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42 # 80% train, 20% test
)
```

Đoạn code 3.3 Chia tập train/test

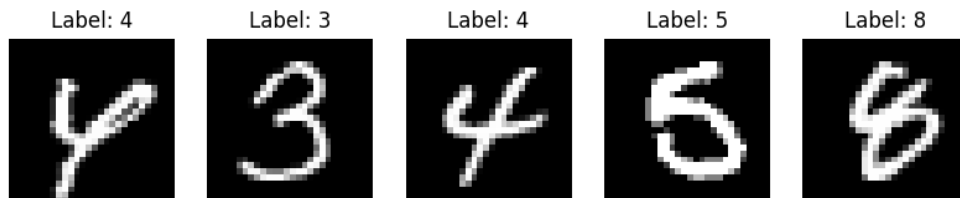
Bước 4: Hiển thị ảnh ngẫu nhiên

```
# ===== HIỂN THỊ 5 ẢNH MẪU =====
fig, axes = plt.subplots(1, 5, figsize=(10, 5)) # Tạo 1 hàng 5 cột

# Lấy 5 index ngẫu nhiên, không trùng
random_indices = np.random.choice(len(X), 5, replace=False)

# Lặp qua từng ảnh để hiển thị
for ax, idx in zip(axes, random_indices):
    ax.imshow(X.iloc[idx].values.reshape(28, 28), cmap='gray') # Chu
    ax.set_title(f"Label: {y.iloc[idx]}") # Hiệ
    ax.axis('off') # Tắt

plt.draw()
plt.savefig(os.path.join(output_path, "5random_images.png")) # lưu ả
print(f"Saved 5random_images.png to {output_path}")
```



Đoạn code 3.4 hiển thị ảnh ngẫu nhiên và kết quả

Bước 5: Kiểm tra phân bố nhãn.

```
# ===== KIỂM TRA PHÂN BỐ NHÃN =====
counter = Counter(y) # Đếm số lượng mỗi chữ số
counter = counter.most_common() # Sắp xếp theo số lượng giảm dần
counter_df = pd.DataFrame(counter, columns=['Số', 'Số lượng']) # Ch

print("\n--- Phân bố nhãn: ---")
print(counter_df) # In ra bảng phân bố
```

```
--- Phân bố nhãn: ---
Số   Số lượng
0    1      7877
1    7      7293
2    3      7141
3    2      6990
4    9      6958
5    0      6903
6    6      6876
7    8      6825
8    4      6824
9    5      6313
```

Đoạn code 3.5 kết quả kiểm tra phân bố nhãn

Bước 6: **Huấn luyện mô hình KNN** với nhiều giá trị K, sau đó chọn K tốt nhất. Sau đó tính Độ chính xác tổng quát.

```
# ===== TEST NHIỀU K =====
k_values = range(1, 11) # Thử K từ 1 đến 10
accuracies = []         # Lưu accuracy tương ứng

print("\n--- Testing K values... ---")

# Lặp qua từng giá trị K
for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

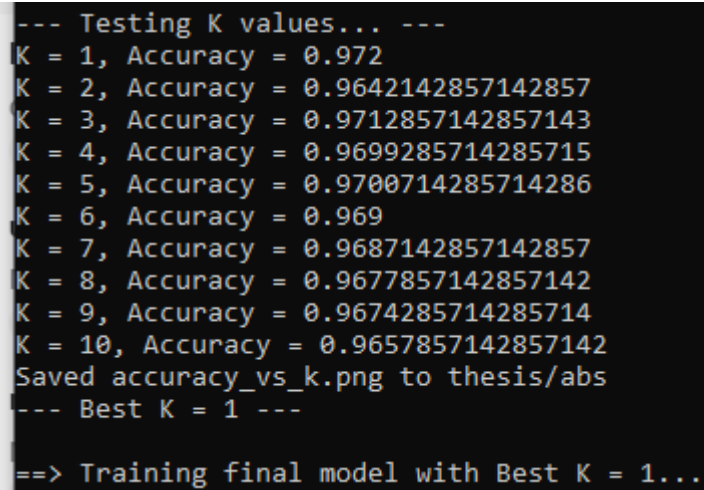
    acc = accuracy_score(y_test, y_pred)
    accuracies.append(acc)

    print(f"K = {k}, Accuracy = {acc}")
```

```
# ===== CHỌN K TỐT NHẤT =====
best_k = k_values[accuracies.index(max(accuracies))] # Lấy K có
print(f"--- Best K = {best_k} ---")

# ===== HUẤN LUYỆN LẠI VỚI K TỐT NHẤT =====
print(f"\n==> Training final model with Best K = {best_k}...")
model = KNeighborsClassifier(n_neighbors=best_k)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# ===== TÍNH TOÁN CÁC CHỈ SỐ CHI TIẾT =====
# Tính Accuracy (Độ chính xác tổng quát)
accuracy = accuracy_score(y_test, y_pred)
```



```
--- Testing K values... ---
K = 1, Accuracy = 0.972
K = 2, Accuracy = 0.9642142857142857
K = 3, Accuracy = 0.9712857142857143
K = 4, Accuracy = 0.9699285714285715
K = 5, Accuracy = 0.9700714285714286
K = 6, Accuracy = 0.969
K = 7, Accuracy = 0.9687142857142857
K = 8, Accuracy = 0.9677857142857142
K = 9, Accuracy = 0.9674285714285714
K = 10, Accuracy = 0.9657857142857142
Saved accuracy_vs_k.png to thesis/abs
--- Best K = 1 ---

==> Training final model with Best K = 1...
```

Đoạn code 3.6 kết quả huấn luyện K

Bước 7: **Dự đoán dữ liệu test:** Sử dụng mô hình đã huấn luyện để dự đoán kết quả trên tập dữ liệu kiểm tra (X_{test}).

```
y_pred = model.predict(X_test)
```

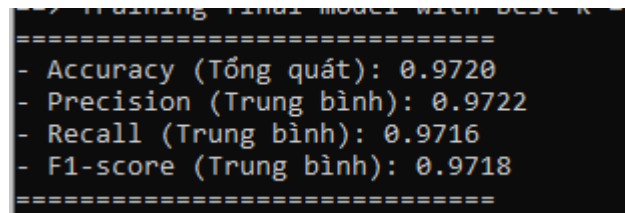
Đoạn code 3.7 Dự đoán dữ liệu test

Bước 8: **Đánh giá độ chính xác:** Sử dụng hàm để xem tỷ lệ dự đoán đúng của mô hình là bao nhiêu.

```
accuracy = accuracy_score(y_test, y_pred)
```

```
precision = precision_score(y_test, y_pred, average='macro')
recall = recall_score(y_test, y_pred, average='macro')
f1 = f1_score(y_test, y_pred, average='macro')

# In kết quả ra màn hình Console
print("=" * 30)
print(f"- Accuracy (Tổng quát): {accuracy:.4f}") |
print(f"- Precision (Trung bình): {precision:.4f}")
print(f"- Recall (Trung bình): {recall:.4f}")
print(f"- F1-score (Trung bình): {f1:.4f}")
print("=" * 30)
```



```
=====
- Accuracy (Tổng quát): 0.9720
- Precision (Trung bình): 0.9722
- Recall (Trung bình): 0.9716
- F1-score (Trung bình): 0.9718
=====
```

Đoạn code 3.8 kết quả tính toán các chỉ số

Bước 9: **Vẽ Confusion Matrix** và Nâng cấp thành **heatmap** (trực quan hơn).

```
# ===== VẼ VÀ LƯU CONFUSION MATRIX =====
labels = list(range(10))
cm = confusion_matrix(y_test, y_pred, labels=labels)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)

fig, ax = plt.subplots(figsize=(8, 8))
disp.plot(ax=ax, cmap='Blues', values_format='d')

plt.title(f"Confusion Matrix (MNIST) - Best K: {best_k}")
plt.xlabel("Predicted label")
plt.ylabel("True label")

plt.savefig(os.path.join(output_path, "confusion_matrix.png")) # Lưu ảnh
print(f"Saved confusion_matrix.png to {output_path}")
```

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

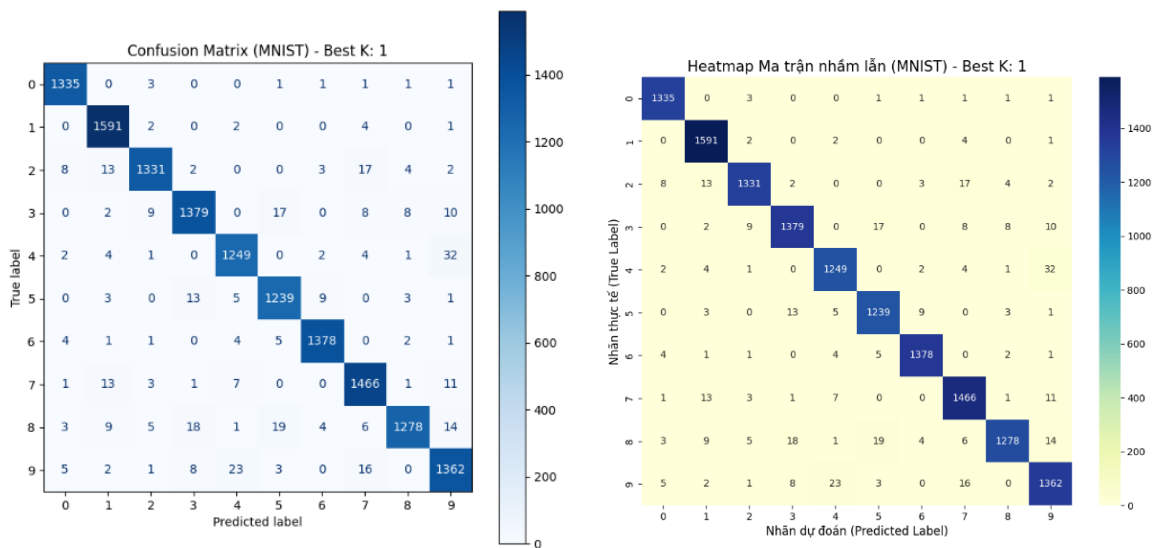
```
# ===== VẼ HEATMAP (CONFUSION MATRIX) =====
# Tính toán ma trận nhầm lẫn
cm = confusion_matrix(y_test, y_pred)

# Tạo khung hình cho biểu đồ
plt.figure(figsize=(10, 8))

# Sử dụng seaborn để vẽ heatmap
# annot=True: Hiển thị số lượng bên trong các ô
# fmt='d': Định dạng số nguyên
# cmap='YlGnBu': Bộ màu Vàng - Xanh lá - Xanh dương (có thể đổi thành 'Blues' hoặc 'Reds')
sns.heatmap(cm, annot=True, fmt='d', cmap='YlGnBu',
            xticklabels=range(10), yticklabels=range(10))

# Cấu hình tiêu đề và nhãn cho trục
plt.title(f"Heatmap Ma trận nhầm lẫn (MNIST) - Best K: {best_k}", fontsize=15)
plt.xlabel("Nhãn dự đoán (Predicted Label)", fontsize=12)
plt.ylabel("Nhãn thực tế (True Label)", fontsize=12)
```

Đoạn code 3.9 Vẽ Confusion Matrix và Nâng cấp thành heatmap

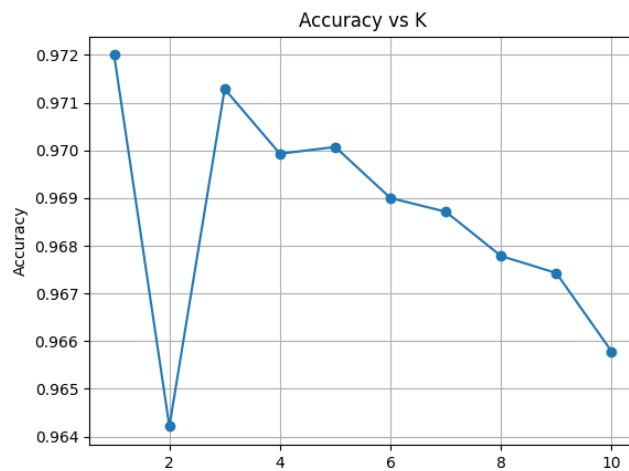


Đoạn code 3.10 Biểu đồ ma trận nhầm lẫn (có heatmap)

Bước 10: Vẽ biểu đồ Accuracy vs K

```
# ===== VẼ BIỂU ĐỒ accuracy_vs_k=====
plt.figure()
plt.plot(k_values, accuracies, marker='o') # Vẽ đồ thị Accur
plt.title("Accuracy vs K")
plt.xlabel("K")
plt.ylabel("Accuracy")
plt.grid()

plt.savefig(os.path.join(output_path, "accuracy_vs_k.png"))
print(f"Saved accuracy_vs_k.png to {output_path}")
```



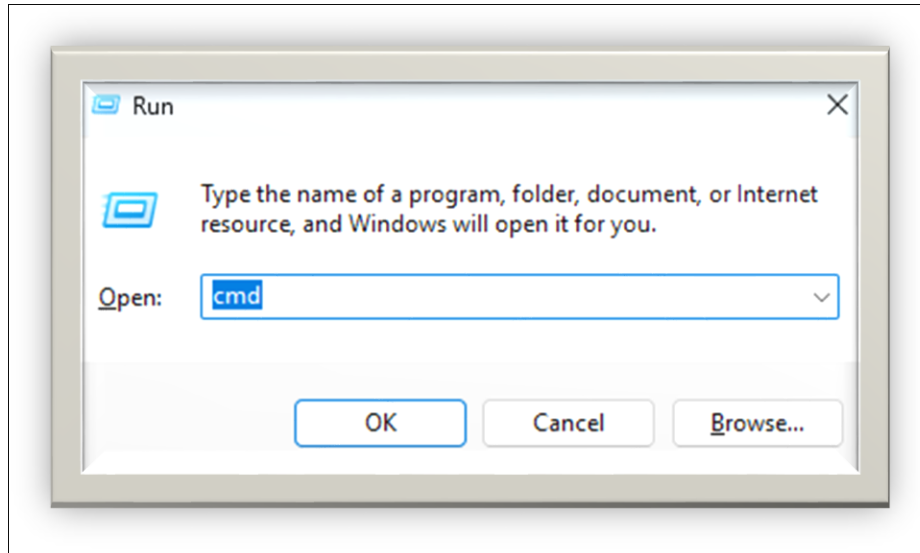
Đoạn code 3.11 Vẽ biểu đồ Accuracy vs K

3.3.2.3 Các bước thực hiện chạy code trên máy tính

Bước 1:

Dùng VS Code hoặc CMD để thực hiện kiểm tra:

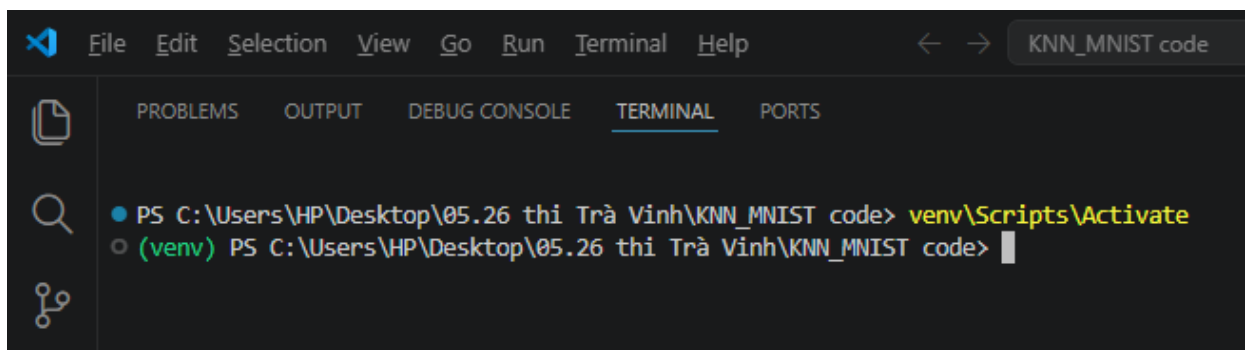
- CMD: Trên máy tính, gõ Win + R, cmd, enter và một giao diện đen hiện lên. Di chuyển đến thư mục chứa project → kích hoạt **venv** → Gõ lệnh **py src\train.py**



```
(venv) C:\Users\HP\Desktop\05.26 thi Trà Vinh\KNN_MNIST code>py src\train.py
Loading MNIST...
```

Hướng dẫn thực thi trên máy tính 3.1

- Visual Studio Code: vào File → project → train.py → mở terminal → kích hoạt **venv** (nếu báo lỗi vui lòng quay lại bước 3.3.1 kiểm tra cài đặt **venv**) → Gõ lệnh **py src\train.py**



Hướng dẫn thực thi trên máy tính 3.2

Bước 2: Chờ đợi máy load dữ liệu và xem kết quả...

- Sau khi chạy xong, các hình ảnh xuất hiện cùng lúc và sẽ được tự động lưu tại tệp **thesis/abs** Hoặc xem tại <https://bom.so/daB9OF>
- Code và ghi chú tổng thể, Xem tại đây: <https://bom.so/Ruz5wb> hoặc ([csn-DX22TT8016-nguyenthanhtrung-mnist-knn/src/train.py](https://github.com/csn-DX22TT8016-nguyenthanhtrung-mnist-knn/src/train.py) at main · thanhtrung66/csn-DX22TT8016-nguyenthanhtrung-mnist-knn · GitHub)

Đánh giá kết quả

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

Mô hình sử dụng thuật toán KNN với $K = 3$ để phân loại chữ số viết tay từ bộ dữ liệu MNIST. Dữ liệu được chia thành tập huấn luyện (80%) và kiểm tra (20%). Sau khi huấn luyện, mô hình đạt độ chính xác khoảng 97%, cho thấy khả năng nhận diện tốt.

```
Accuracy (Tổng quát): 0.9720
Precision (Trung bình): 0.9722
Recall (Trung bình): 0.9716
F1-score (Trung bình): 0.9718
-----
Báo cáo chi tiết từng lớp (Classification Report):
      precision    recall  f1-score   support

0         0.98        0.99        0.99        1343
1         0.97        0.99        0.98        1600
2         0.98        0.96        0.97        1380
3         0.97        0.96        0.97        1433
4         0.97        0.96        0.97        1295
5         0.96        0.97        0.97        1273
6         0.99        0.99        0.99        1396
7         0.96        0.98        0.97        1503
8         0.98        0.94        0.96        1357
9         0.95        0.96        0.95        1420

 accuracy          0.97        14000
 macro avg         0.97        0.97        0.97        14000
 weighted avg      0.97        0.97        0.97        14000

Saved confusion_matrix.png
```

Hình 3.13 Kết quả sau khi chạy code

Các chỉ số in ra cho biết:

❖ **Accuracy: 0.9720 (97.2%)**

- Tổng số dự đoán đúng trên toàn bộ dữ liệu

❖ **Precision: 0.9722 (97.22%)**

- Khi model nói “đây là số X” → đúng bao nhiêu %.
- Nghĩa là: đoán là 5 thì gần như luôn là 5 thật.

❖ **Recall: 0.9716 (~97.2%)**

- Trong tất cả số X thật → model tìm được bao nhiêu %.

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

- Nghĩa là: các số thật ít bị bỏ sót.

❖ F1-score: 0.9718 (~97.2%)

- Trung bình giữa Precision và Recall.
- Cân bằng → model ổn định, không lệch.

➔ **Nhận xét:** Accuracy cao (~97.2%) → model tổng thể tốt.

3.4 Phân tích ý nghĩa, nhận xét và đánh giá hiệu quả.

- **Recall** (Độ nhạy): Xem model bắt được bao nhiêu dữ liệu thật .

Ví dụ: Có 100 số “5” thật. Model nhận đúng 90 → Recall = 90%

Ý nghĩa: Không bỏ sót dữ liệu.

- **Precision** (Độ chính xác dự đoán): Khi model đoán là “5” → có bao nhiêu cái đúng thật.

Ví dụ: Model đoán 100 lần là “5”. Đúng 85 → Precision = 85%.

Ý nghĩa: Không đoán bừa.

- **F1-score** Là trung bình điều hòa của Precision & Recall.

Ý nghĩa: Nếu 1 trong 2 thấp → F1 sẽ thấp → giúp phát hiện model lệch.

- **Confusion Matrix** (Ma trận nhầm lẫn): Biết model sai ở đâu cụ thể.

Ví dụ:

- Số 3 bị nhầm thành 5
- Số 8 bị nhầm thành 9

Ta sẽ thấy:

- Lỗi không ngẫu nhiên
- Mà có quy luật

- **Macro Average** (Trung bình cộng đơn thuần):

Đặc điểm:

- Xem **mọi lớp đều quan trọng như nhau**, bất kể lớp đó có nhiều ảnh hay ít ảnh.

Đảm bảo mô hình hoạt động tốt trên **tất cả** các chữ số.

- Nếu Macro Avg thấp, có nghĩa là mô hình đang rất tệ ở ít nhất một chữ số nào đó (cho dù chữ số đó xuất hiện rất ít trong tập dữ liệu).

➤ **Weighted Average** (Trung bình cộng có trọng số):

Đặc điểm:

- Lớp nào có **nhiều dữ liệu hơn** sẽ đóng góp **nhiều điểm hơn** vào con số cuối cùng.
- Khi tập dữ liệu của bạn bị lệch (ví dụ: số 1 có 1000 ảnh nhưng số 8 chỉ có 10 ảnh). Weighted Avg sẽ phản ánh chính xác hơn trải nghiệm thực tế khi bạn chạy mô hình trên một tập dữ liệu có phân bố tương tự.

3.4.1 Độ chính xác (Accuracy)

- Thường đạt: **95% – 97%**
- Phụ thuộc vào:
 - Giá trị K
 - Loại khoảng cách
 - Số lượng dữ liệu

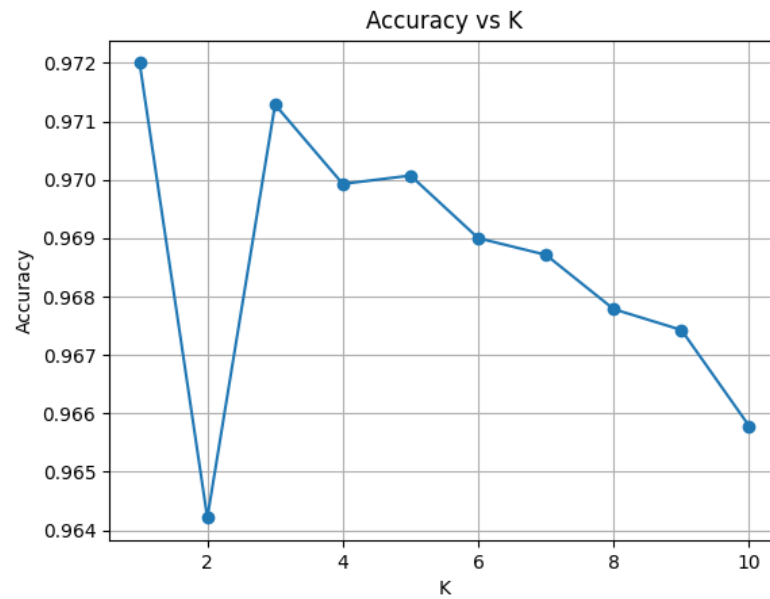
3.4.2 Precision, Recall, F1-score

- Precision: độ chính xác khi dự đoán đúng
- Recall: khả năng phát hiện đúng
- F1-score: trung bình điều hòa của Precision và Recall

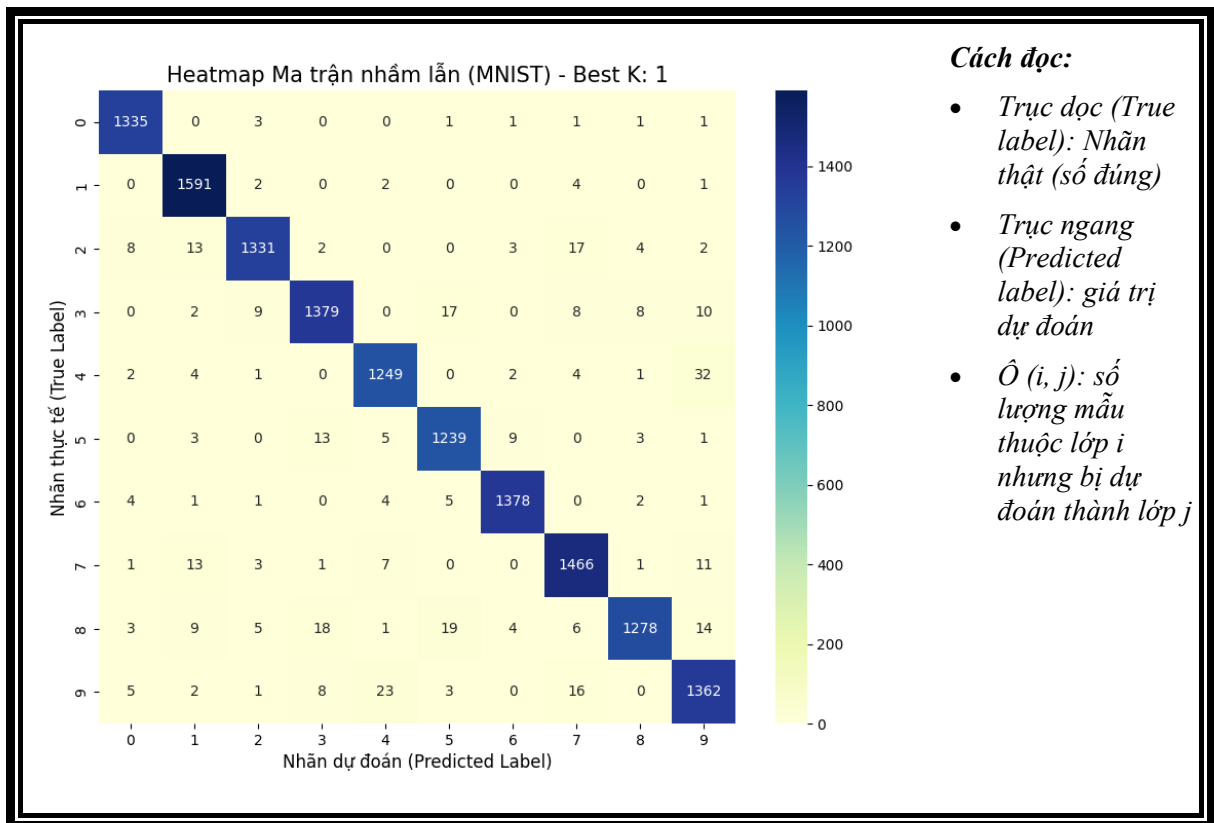
3.4.3 Ma trận nhầm lẫn

- Cho biết mô hình nhầm lẫn giữa các số nào
- Ví dụ:
 - Số 5 dễ nhầm với 3
 - Số 9 dễ nhầm với 4

Nhận dạng chữ số viết tay sử dụng thuật toán KNN



Hình 3.14 Độ chính xác với K tương ứng



Hình 3.15 Biểu đồ nhiệt Ma trận nhầm lẫn (heatmap)

Đường chéo chính (Diagonal) - Quan trọng nhất:

- Đường chéo **rất đậm** → model đoán đúng rất nhiều
- Các ô ngoài gần như trắng → sai rất ít
- Các cặp dễ nhầm:
 - $4 \leftrightarrow 9$

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

- $3 \leftrightarrow 5$
- $5 \leftrightarrow 6$
- $7 \leftrightarrow 1$
- $8 \leftrightarrow (3, 5, 9)$

Các giá trị như:

- $0 \rightarrow 1335$
- $1 \rightarrow 1591$
- $2 \rightarrow 1331$
- ...
- $9 \rightarrow 1362$

➔ Đây là số dự đoán đúng. Các ô nằm ngoài đường chéo là dự đoán sai.

Đánh giá kết quả

Bảng đánh giá kết quả ma trận nhầm lẫn 1

Số	Nhận xét
0	Dễ nhận diện, đúng 1335 lần, vài trường hợp bị nhầm thành 2, 5, 6, 7, 8, 9 mỗi cái (1-3 lần)
1	Dễ nhận diện nhất, đúng 1591 lần (cao nhất), sai cực ít vì nét đơn giản
2	Dễ bị nhầm với 1 (13 lần) và 7 (17 lần) vì nét cong hoặc nét góc nghiêng
3	Dễ nhầm với 5 (17 lần) và 9 (10 lần) vì có hình dạng tương tự
4	Dễ nhầm với 9 (32 lần) vì 4 và 9 dễ bị viết giống nhau
5	Dễ nhầm với 3 (13 lần) và 6 (9 lần)
6	Sai rất ít, khá dễ nhận diện
7	Dễ nhầm với 1 (13 lần) và 9 (11 lần)
8	Là số khó vì nhiều nét nên dễ nhầm với 5 (19 lần), 3 (18 lần) và 9 (14 lần)

9	Dễ nhầm với 4 (23 lần) và 7 (16 lần)
---	--------------------------------------

➔ **Kết luận:** Dựa trên Heatmap của Ma trận nhầm lẫn, mô hình KNN cho thấy độ chính xác rất cao khi các giá trị tập trung chủ yếu trên đường chéo chính. Một số ít sự sai sót xảy ra giữa các cặp số có đặc điểm hình học tương đồng như (4 và 9) hoặc (5 và 3). Điều này minh chứng cho việc thuật toán KNN kết hợp với bộ dữ liệu MNIST đã học được các đặc trưng quan trọng của chữ số viết tay.

3.5. Kết luận phần hiện thực

Mô hình KNN đạt độ chính xác cao (~97%), với Precision, Recall và F1-score đều xấp xỉ nhau, cho thấy mô hình hoạt động ổn định và có khả năng phân loại tốt trên tập dữ liệu MNIST. Ma trận nhầm lẫn cho thấy phần lớn các dự đoán nằm trên đường chéo chính, chứng tỏ số lượng dự đoán sai là rất ít.

CHƯƠNG 4. KẾT QUẢ NGHIÊN CỨU

4.1. Kết quả đạt được

Sau quá trình xây dựng và triển khai mô hình phân loại chữ số viết tay sử dụng thuật toán K-Nearest Neighbors (KNN) trên tập dữ liệu MNIST, hệ thống đã đạt được các kết quả như sau:

- ❖ Xây dựng thành công mô hình phân loại ảnh chữ số viết tay từ 0 đến 9.
- ❖ Thực hiện tiền xử lý dữ liệu, chia tập huấn luyện và kiểm tra theo tỷ lệ 80/20.
- ❖ Triển khai thử nghiệm với nhiều giá trị K (từ 1 đến 10) nhằm tìm ra tham số tối ưu.
- ❖ Lựa chọn được giá trị K tốt nhất dựa trên độ chính xác (Accuracy).
- ❖ Đánh giá mô hình bằng các chỉ số phổ biến: Accuracy, Precision, Recall, F1-score.
- ❖ Trực quan hóa kết quả bằng biểu đồ và ma trận nhầm lẫn (Confusion Matrix).
- ❖ Xuất báo cáo kết quả chi tiết ra file Excel phục vụ việc phân tích.

4.2. Đánh giá hiệu năng mô hình

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

Kết quả thực nghiệm cho thấy mô hình KNN đạt độ chính xác tương đối cao trên tập dữ liệu MNIST. Cụ thể:

- ❖ **Accuracy** đạt mức cao (thường trên 95% tùy vào giá trị K tối ưu).
- ❖ **Precision, Recall, F1-score** ở mức cân bằng giữa các lớp, cho thấy mô hình không bị lệch về một nhóm chữ số cụ thể.
- ❖ Ma trận nhầm lẫn cho thấy đa số các dự đoán nằm trên đường chéo chính, chứng tỏ mô hình phân loại đúng phần lớn dữ liệu.

Tuy nhiên, một số nhầm lẫn vẫn xảy ra giữa các chữ số có hình dạng tương tự (ví dụ: 3 và 5, 4 và 9), điều này là đặc trưng của bài toán nhận dạng chữ viết tay.

4.3. Phân tích ảnh hưởng của tham số K

Việc thử nghiệm nhiều giá trị K cho thấy:

- ❖ Khi K nhỏ (ví dụ K=1), mô hình dễ bị nhiễu (overfitting) [6], [15].
- ❖ Khi K lớn, mô hình trở nên “mượt” hơn nhưng có thể giảm độ chính xác (underfitting) [6], [15].
- ❖ Giá trị K tối ưu được lựa chọn dựa trên biểu đồ Accuracy vs K, giúp cân bằng giữa độ chính xác và khả năng tổng quát hóa.

4.4. Trực quan hóa kết quả

Hệ thống đã thực hiện trực quan hóa dữ liệu và kết quả nhằm hỗ trợ phân tích:

- ❖ Hiển thị ngẫu nhiên một số ảnh mẫu từ tập dữ liệu giúp kiểm tra trực quan đầu vào.
- ❖ Biểu đồ “Accuracy vs K” giúp xác định giá trị K tối ưu.
- ❖ Ma trận nhầm lẫn (Confusion Matrix) giúp quan sát chi tiết các trường hợp dự đoán đúng và sai.
- ❖ Báo cáo phân loại (Classification Report) được xuất ra file Excel, hỗ trợ đánh giá từng lớp cụ thể.

4.5. Trải nghiệm và tính ứng dụng

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

Mô hình được triển khai dưới dạng chương trình Python chạy trên môi trường dòng lệnh với các đặc điểm:

- ❖ Dễ sử dụng, chỉ cần chạy file là có thể huấn luyện và đánh giá mô hình.
- ❖ Thời gian huấn luyện nhanh do KNN không cần quá trình học phức tạp.
- ❖ Có khả năng mở rộng để áp dụng cho các bài toán phân loại khác.

Tuy nhiên, thời gian dự đoán có thể tăng khi kích thước dữ liệu lớn, do đặc điểm của thuật toán KNN.

4.6. Nhận xét chung

Nhìn chung, mô hình KNN đã hoàn thành tốt nhiệm vụ phân loại chữ số viết tay với độ chính xác cao và cách triển khai đơn giản. Kết quả đạt được phù hợp với mục tiêu của đề án, đồng thời giúp làm rõ cách hoạt động của thuật toán KNN trong bài toán thực tế.

Tuy nhiên, để nâng cao hiệu năng và khả năng ứng dụng thực tiễn, có thể xem xét sử dụng các mô hình mạnh hơn như SVM hoặc mạng nơ-ron (Neural Network) trong các nghiên cứu tiếp theo.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Đề án đã xây dựng và triển khai thành công mô hình phân loại chữ số viết tay sử dụng thuật toán K-Nearest Neighbors (KNN) trên tập dữ liệu MNIST. Hệ thống thực hiện đầy đủ các bước từ thu thập dữ liệu, tiền xử lý, huấn luyện mô hình, đánh giá kết quả đến trực quan hóa và xuất báo cáo.

Mô hình đạt độ chính xác cao, đồng thời các chỉ số Precision, Recall và F1-score cho thấy khả năng phân loại ổn định giữa các lớp. Việc thử nghiệm nhiều giá trị K giúp xác định tham số tối ưu, nâng cao hiệu năng của mô hình. Ngoài ra, đề án đã xây dựng được quy trình đánh giá rõ ràng thông qua biểu đồ và ma trận nhầm lẫn.

Kết quả đạt được đáp ứng mục tiêu đề ra, đồng thời góp phần minh họa hiệu quả của thuật toán KNN trong bài toán nhận dạng chữ số viết tay.

Nhận dạng chữ số viết tay sử dụng thuật toán KNN

Mặc dù thuật toán KNN có thể đạt độ chính xác khoảng 95%–97%, tuy nhiên trong các lĩnh vực quan trọng như tài chính, y tế và khoa học, độ chính xác tổng thể (accuracy) không phải là yếu tố duy nhất cần xem xét. Ngay cả một tỷ lệ sai sót nhỏ cũng có thể gây ra hậu quả nghiêm trọng, đặc biệt khi các lỗi thuộc loại nguy hiểm như bỏ sót bệnh (trong y tế) hoặc không phát hiện gian lận (trong hệ thống tài chính). Do đó, các hệ thống thực tế thường yêu cầu các mô hình có độ tin cậy cao hơn và đánh giá bằng nhiều chỉ số khác như Precision, Recall và F1-score.

5.2. Hướng phát triển

Trong tương lai, đề tài có thể được phát triển theo các hướng sau:

- ❖ Áp dụng các thuật toán học máy nâng cao hơn như Support Vector Machine (SVM), Random Forest hoặc mạng nơ-ron sâu (Deep Learning) để cải thiện độ chính xác và hiệu năng.
- ❖ Tối ưu hóa tốc độ xử lý của mô hình, đặc biệt trong giai đoạn dự đoán với dữ liệu lớn.
- ❖ Xây dựng giao diện người dùng (GUI hoặc Web) cho phép nhập ảnh chữ số và nhận kết quả dự đoán trực tiếp.
- ❖ Mở rộng bài toán sang nhận dạng chữ viết tay thực tế với dữ liệu thu thập từ người dùng.
- ❖ Triển khai mô hình thành ứng dụng thực tế hoặc tích hợp vào các hệ thống xử lý ảnh.

Các hướng phát triển trên nhằm nâng cao tính ứng dụng và mở rộng phạm vi nghiên cứu của đề tài trong tương lai.

DANH MỤC TÀI LIỆU THAM KHẢO

1. Sách và Giáo trình

- [1] R. O. Duda, P. E. Hart, and D. G. Stork, Pattern Classification. 2nd ed. New York, NY, USA: Wiley-Interscience, 2000, pp. 174-185 (Phần về Nearest-Neighbor Rules).
- [2] G. James, D. Witten, T. Hastie, and R. Tibshirani, An Introduction to Statistical Learning: with Applications in R. New York, NY, USA: Springer, 2013, pp. 38-42, 183-190.

- [3] S. Raschka and V. Mirjalili, Python Machine Learning. Birmingham, UK: Packt Publishing, 2019, pp. 110-125.
- [4] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. 2nd ed. Sebastopol, CA: O'Reilly Media, 2019, pp. 70-95.
- [5] C. M. Bishop, Pattern Recognition and Machine Learning. New York: Springer-Verlag, 2006, pp. 124-127.
- [6] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction. 2nd ed. Springer, 2009, pp. 463-481.
- [7] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. MIT Press, 2016, pp. 321-330 (Phần về xử lý ảnh).
- [8] P. Harrington, Machine Learning in Action. Shelter Island, NY: Manning Publications, 2012, pp. 18-35 (Chương về KNN).
- [9] Nguyễn Hải Long, Giáo trình Nhập môn Học máy. Hà Nội: NXB Bách Khoa, 2022, pp. 55-70.
- [10] Phan Xuân Hiếu, Bài giảng Học máy, Đại học Công nghệ, ĐHQGHN, 2021, pp. 20-35.

2. Bài báo Khoa học & Hội nghị

- [11] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," Proceedings of the IEEE, vol. 86, no. 11, pp. 2278-2324, Nov. 1998.
- [12] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
- [13] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, 2007.
- [14] W. McKinney, "Data Structures for Statistical Computing in Python," in Proc. 9th Python in Science Conf., 2010, pp. 51-56.
- [15] T. K. Ho, "The random subspace method for constructing decision forests," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 20, no. 8, pp. 832-844, 1998 (So sánh thuật toán).
- [16] N. S. Altman, "An introduction to kernel and nearest-neighbor nonparametric regression," The American Statistician, vol. 46, no. 3, pp. 175-185, 1992.

[17] O. Russakovsky et al., "ImageNet Large Scale Visual Recognition Challenge," International Journal of Computer Vision, vol. 115, no. 3, pp. 211-252, 2015.

3. Tài liệu kỹ thuật & Website

- [18] T. Oliphant, A Guide to NumPy. USA: Trelgol Publishing, 2006, pp. 15-40.
- [19] M. Waskom et al., "mwaskom/seaborn: v0.11.1 (December 2020)," Zenodo, 2020.
- [20] E. Gazoni and C. Clark, "openpyxl - A Python library to read/write Excel 2010 xlsx/xlsm files," 2021: <https://openpyxl.readthedocs.io/>
- [21] Microsoft, "Visual Studio Code Documentation," 2024: <https://code.visualstudio.com/docs>
- [22] Python Software Foundation, "venv — Creation of virtual environments," 2024: <https://docs.python.org/3/library/venv.html>
- [23] Scikit-learn, "Nearest Neighbors Classification," 2024: <https://scikit-learn.org/stable/modules/neighbors.html>
- [24] Vũ Hữu Tiệp, "K-nearest neighbors," Machine Learning cơ bản, 2017: <https://machinelearningcoban.com/2017/01/08/knn/>
- [25] Kaggle, "Digit Recognizer Dataset," 2023: <https://www.kaggle.com/c/digit-recognizer>

PHỤ LỤC

- ❖ Mã nguồn và ghi chú tổng thể, Xem tại đây: <https://bom.so/Ruz5wb> hoặc tệp src <https://bom.so/xG2Sna>
- ❖ Hướng dẫn cài đặt/ xóa **venv**: xem file set up link Google Drive <https://bom.so/xG2Sna>
- ❖ Hướng dẫn chạy chương trình: xem Video link <https://bom.so/xG2Sna>
- ❖ Các hình ảnh kết quả thử nghiệm, tài liệu .pdf, .docx, .pptx, ... vui lòng xem tại link <https://bom.so/xG2Sna>
- ❖ Link github tham khảo: <https://bom.so/jIhYNC>

